

CM10 - Dynamic Memory, Structs, Lists, Hash Tables

Heap allocation, user-defined types, linked lists, hash-table dictionary

Louis Ledoux

2026-07-05

Table of contents

1	Dynamic Memory Allocation	3
1.1	Main Path	3
1.2	Worked Example	4
1.3	Anecdote Or Motivation	4
1.4	Check Question	5
1.5	Backup Index	5
1.6	Backup: CM10 Dynamic memory allocation	8
1.7	Backup: Variable allocation in memory	8
1.8	Backup: Static & Automatic memory allocation	9
1.9	Backup: Limitations & inefficiencies	10
1.10	Backup: Dynamic memory allocation	11
1.11	Backup: Memory Leak	11
1.12	Backup: Memory Leak	12
1.13	Backup: Dangling pointer	12
1.14	Backup: Dynamic memory: Functions	13
1.15	Backup: Dynamic allocation: malloc()	14
1.16	Backup: Dynamic de-allocation: free(...)	15
1.17	Backup: Dynamic allocation: Example	16
1.18	Backup: Dynamic Memory Allocation	16
1.19	Backup: 1D Array with size elements	17
1.20	Backup: 1D Array with size characters -> string	18
1.21	Backup: String declaration: Remarks	19
1.22	Backup: 2D Array: Single pointer	20
1.23	Backup: 2D Array: Array of pointers	21
1.24	Backup: 2D Array: Pointer to pointer	22
1.25	Backup: Dynamic Memory Allocation	23
2	User-defined Data Types	24
2.1	Main Path	24
2.2	Worked Example	25
2.3	Anecdote Or Motivation	25
2.4	Check Question	26
2.5	Backup Index	26
2.6	Backup: User-defined data types: Enum, typedef, struct	29
2.7	Backup: Enumerated type declaration & definition	29
2.8	Backup: Structure type declaration	30
2.9	Backup: Structure type declaration	31
2.10	Backup: Structure variable definition & initialisation	31
2.11	Backup: Structure padding	32
2.12	Backup: Structure padding: Example	33

2.13 Backup: Structure variable: Access members	34
2.14 Backup: Definition of a new type	35
2.15 Backup: Structure type: Examples	36
2.16 Backup: Access struct members	36
2.17 Backup: Structures	37
3 Lists	37
3.1 Main Path	38
3.2 Worked Example	38
3.3 Anecdote Or Motivation	39
3.4 Check Question	39
3.5 Backup Index	40
3.6 Backup: Data structures: Lists and Hash tables	42
3.7 Backup: Linked lists Vs arrays	43
3.8 Backup: Why Linked list?	44
3.9 Backup: Linked list	45
3.10 Backup: Linked list: Example	46
3.11 Backup: Linked list: Example	47
3.12 Backup: Linked list: Example	48
3.13 Backup: Linked list: Example	49
3.14 Backup: Linked list: Example	51
3.15 Backup: Linked list: Example	52
3.16 Backup: Linked list: Example	53
3.17 Backup: Why pass by reference?	54
3.18 Backup: Why pass by reference?	56
3.19 Backup: Why pass by reference?	57
3.20 Backup: Why pass by reference?	59
3.21 Backup: Operations: Insertion	60
3.22 Backup: Operations: Insertion	61
3.23 Backup: Operations: Insertion	62
3.24 Backup: Operations: Insertion	63
3.25 Backup: Operations: Insertion	64
3.26 Backup: Operations: Insertion	65
3.27 Backup: Operations: Insertion	66
3.28 Backup: Operations: Insertion	67
3.29 Backup: Operations: Insertion	68
3.30 Backup: Operations: Insertion	69
3.31 Backup: Operations: Insertion	70
3.32 Backup: Operations: Insertion	71
3.33 Backup: Operations: Insertion	72
3.34 Backup: Operations: Insertion	73
3.35 Backup: Operations: Insertion	74
3.36 Backup: Operations: Insertion	75
3.37 Backup: Operations: Insertion	76
3.38 Backup: Operations: Insertion	77
3.39 Backup: Operations: Insertion	78
3.40 Backup: Operations: Insertion	79
3.41 Backup: Operations: Deletion	80
3.42 Backup: Operations: Deletion	81
3.43 Backup: Operations: Deletion	82
3.44 Backup: Operations: Insertion	83
3.45 Backup: Operations: Insertion	84
3.46 Backup: Operations: Insertion	85

3.47 Backup: Operations: Insertion	86
3.48 Backup: Doubly linked list	87
4 Hash Tables	88
4.1 Main Path	88
4.2 Worked Example	89
4.3 Anecdote Or Motivation	89
4.4 Check Question	90
4.5 Backup Index	90
4.6 Backup: Hash Table	93
4.7 Backup: Hash Table: Some of its use	94
4.8 Backup: Hash Function: Simple examples	94
4.9 Backup: Collisions	95
4.10 Backup: Collisions: Solution	96
4.11 Backup: Problems	97
4.12 Backup: Hash table: Dictionary	98
4.13 Backup: Hash table: Dictionary	98
4.14 Backup: Hash table: Dictionary	99
4.15 Backup: Hash table: Dictionary	100
4.16 Backup: Hash table: Dictionary	101
4.17 Backup: Hash table: Dictionary	102
4.18 Backup: Hash table: Dictionary	102
4.19 Backup: Hash table: Dictionary	103
4.20 Backup: Hash table: Dictionary	104
4.21 Backup: Hash table: Dictionary	105

1 Dynamic Memory Allocation

Live pacing: about 14 min

Question. When stack objects are not enough, who owns memory and structure?

Core claim. Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.

Target capability. Students can pair allocation/free, define structs, and reason about linked/hash structures.

1.1 Main Path

- Start from the question: When stack objects are not enough, who owns memory and structure?
- Build the model: Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.
- End with a checkable action: Students can pair allocation/free, define structs, and reason about linked/hash structures.
- Keep only this slide on the horizontal path during the live lecture.

- CM10 Dynamic memory allocation

#281

1.2 Worked Example

- Use this as the live trace: Allocate one node, link it into a list, then remove it and state exactly who frees it.
- Representative source slide: 289
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Dynamic memory: Functions

```
Directive:  
#include <stdlib.h>  
Allocation
```

#289

1.3 Anecdote Or Motivation

Every `malloc` creates a future obligation. Good C code makes that obligation visible.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

1.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Dynamic Memory Allocation

- Kahoot time!
- Q6 - Q8

#300

1.5 Backup Index

Original slides 281-300

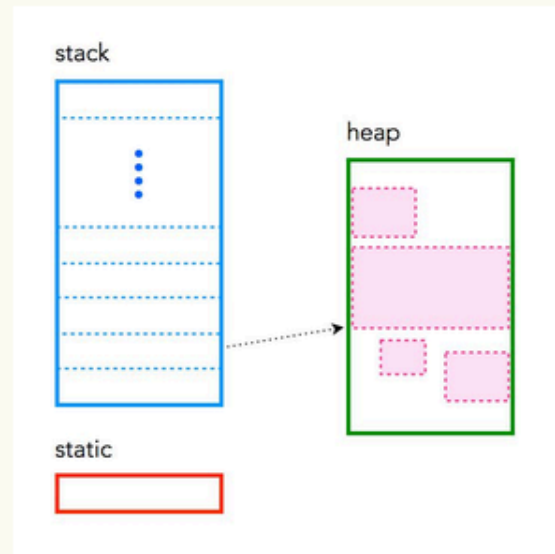
- CM10 Dynamic memory allocation

#281

slide 281

Variable allocation in memory

- Static:
 - global and static variable storage
 - permanent for the entire run of the program.
 - Allocation at compile/link time
- Stack:
 - local variable storage
 - automatic, continuous memory
 - Allocation during execution
- Heap:
 - dynamic storage
 - large pool of memory, not allocated in contiguous order
 - Allocation during execution



#282

slide 282

Static & Automatic memory allocation

- Static:
 - allocated when the program starts
 - Variable lifetime is the entire program
- Automatic:
 - allocated during execution (function calls)
 - managed by the OS and the compiler
 - Variable lifetime is the block where they are defined
 - The variable size is fixed
 - Usually stored on the stack

#283

slide 283

Limitations & inefficiencies

- Not always possible to know the exact number of array elements. May depend:
- Parameters of program execution
- Information provided by the user during execution
- Limited control over the lifetime of the variables
- Worst case: Stack overflow
- Example: Recursive functions

#284

slide 284

Dynamic memory allocation

- The memory to store the program variables is allocated during the execution of the program
- Stored in the heap (large free pool of memory)
- The programmer controls:

#285

slide 285

Memory Leak

- Allocated blocks in the heap that are now unused and unreferenced !

	- Heap
- 0x1000	
- 0x1004	
- 0x1008	
- 0x100C	
- 0x1014	- 10
- 0x1018	- 20
	- 30
	- 40

#286

slide 286

Anchors: CM10 Dynamic memory allocation; Variable allocation in memory; Static & Automatic memory allocation; Limitations & inefficiencies; Dynamic memory allocation; Memory Leak

1.6 Backup: CM10 Dynamic memory allocation

Original slide 281

- CM10 Dynamic memory allocation

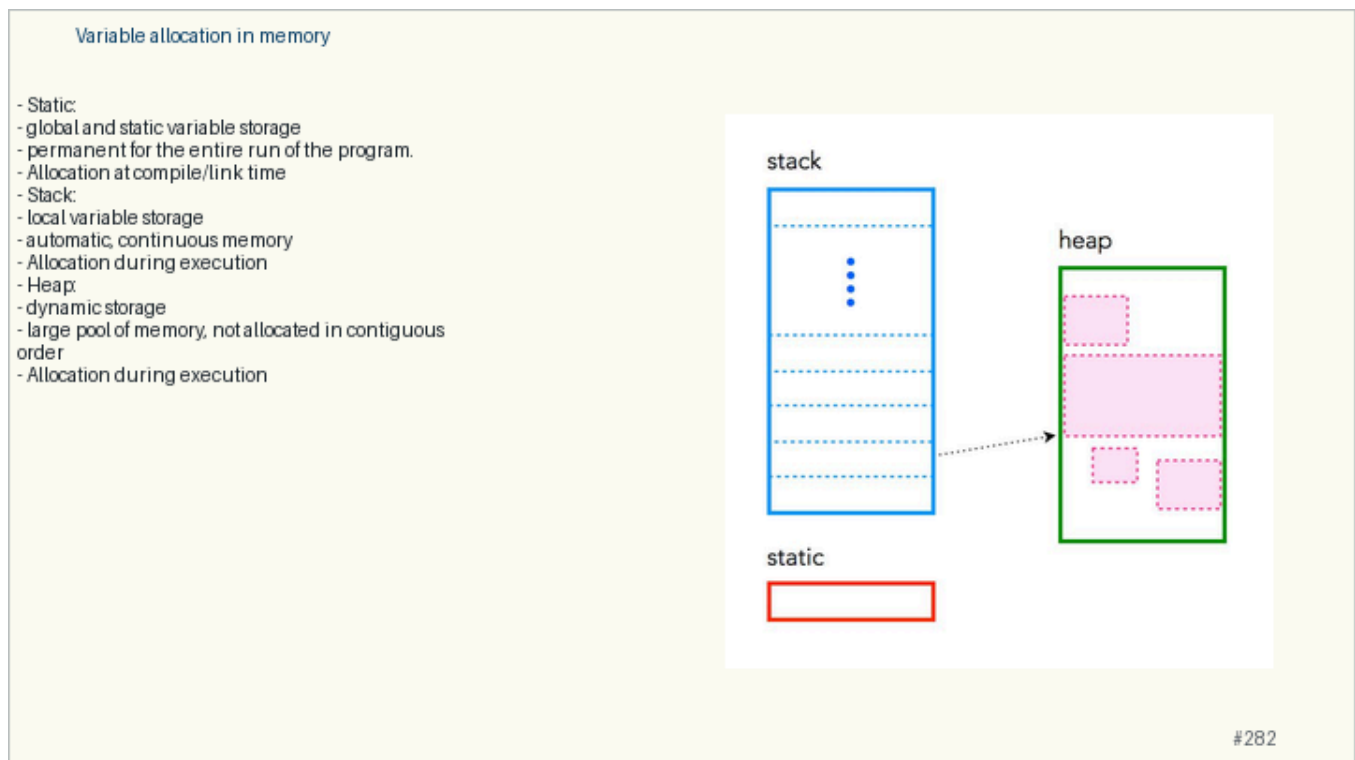
#281

1.7 Backup: Variable allocation in memory

Original slide 282

- Static:
- global and static variable storage

- permanent for the entire run of the program.
- Allocation at compile/link time
- Stack:
 - local variable storage
 - automatic, continuous memory
 - Allocation during execution
- Heap:
 - dynamic storage
 - large pool of memory, not allocated in contiguous order
 - Allocation during execution



1.8 Backup: Static & Automatic memory allocation

Original slide 283

- Static:
 - allocated when the program starts
 - Variable lifetime is the entire program
- Automatic:
 - allocated during execution (function calls)
 - managed by the OS and the compiler
 - Variable lifetime is the block where they are defined
 - The variable size is fixed
 - Usually stored on the stack

Static & Automatic memory allocation

- Static:
 - allocated when the program starts
 - Variable lifetime is the entire program
- Automatic:
 - allocated during execution (function calls)
 - managed by the OS and the compiler
 - Variable lifetime is the block where they are defined
 - The variable size is fixed
 - Usually stored on the stack

#283

1.9 Backup: Limitations & inefficiencies

Original slide 284

- Not always possible to know the exact number of array elements. May depend:
- Parameters of program execution
- Information provided by the user during execution
- Limited control over the lifetime of the variables
- Worst case: Stack overflow
- Example: Recursive functions

Limitations & inefficiencies

- Not always possible to know the exact number of array elements. May depend:
- Parameters of program execution
- Information provided by the user during execution
- Limited control over the lifetime of the variables
- Worst case: Stack overflow
- Example: Recursive functions

#284

1.10 Backup: Dynamic memory allocation

Original slide 285

- The memory to store the program variables is allocated during the execution of the program
- Stored in the heap (large free pool of memory)
- The programmer controls:
 - the exact size (instantiated during execution)
 - Lifetime of memory locations
- Worst-case: Memory leaks!
- Problems: Dangling pointers!
- ...

Dynamic memory allocation

- The memory to store the program variables is allocated during the execution of the program
- Stored in the heap (large free pool of memory)
- The programmer controls:

#285

1.11 Backup: Memory Leak

Original slide 286

- Allocated blocks in the heap that are now unused and unreferenced !
- Heap
- 0x1000
- 0x1004
- 0x1008
- 0x100C
- 0x1014
- 10
- 0x1018
- 20
- 30
- 40

Memory Leak

- Allocated blocks in the heap that are now unused and unreferenced !

	- Heap
- 0x1000	
- 0x1004	
- 0x1008	
- 0x100C	
- 0x1014	- 10
- 0x1018	- 20
	- 30
	- 40

#286

1.12 Backup: Memory Leak

Original slide 287

- Improper use of dynamic memory -> causes the memory consumption to be increased with the time
- In JAVA, garbage collector takes care of the not removed reserved parts in the heap !
- In C, the programmer is responsible for that!
- For EACH allocation -> We NEED a DEALLOCATION

Memory Leak

- Improper use of dynamic memory -> causes the memory consumption to be increased with the time
- In JAVA, garbage collector takes care of the not removed reserved parts in the heap !
- In C, the programmer is responsible for that!

- For EACH allocation -> We NEED a DEALLOCATION

#287

1.13 Backup: Dangling pointer

Original slide 288

- An object is deleted or de-allocated
- If the value of the pointer is not modified
- the pointer still points to the memory location of the deleted/de-allocated memory.
- Heap/Stack
- 0x100C
- 0x1014
- 10
- 0x1018
- 20
- 30
- 40

Dangling pointer

- An object is deleted or de-allocated
- If the value of the pointer is not modified
- the pointer still points to the memory location of the deleted/de-allocated memory.

- Heap/Stack

- 0x100C

- 0x1014

- 0x1018

- 10

- 20

- 30

- 40

#288

1.14 Backup: Dynamic memory: Functions

Original slide 289

Directive:

```
#include <stdlib.h>
```

Allocation

```
malloc()
```

```
calloc()
```

```
realloc()
```

Deallocation:

```
free()
```

Dynamic memory: Functions

Directive:
#include <stdlib.h>
Allocation

#289

1.15 Backup: Dynamic allocation: malloc()

Original slide 290

Syntax:

```
void *malloc(size_t size);
```

Function

Reserves a block with a size equal to size bytes in the heap

size_t is **unsigned** integer type of at least **16** bit

The content of the memory block it is NOT initialized

To calculate the required memory size to store an object we should use the function **sizeof**(type)

The actual size of data object is compiler and target machine depended

Return value:

pointer of **void** type with the address of the first reserved byte (**void***)

We have to explicitly cast the pointer to the type of data we want to store(**int***, **char***, ...)

Dynamic allocation: malloc()

```
void *malloc(size_t size);  
Function
```

#290

1.16 Backup: Dynamic de-allocation: free(...)

Original slide 291

Syntax:

```
void free(void *pointer);
```

Function

free() frees the memory block pointed out by the pointer, which has been reserved by a call, such as malloc(), calloc() ou realloc()

If the pointer has not been obtained by one of these functions, or it has been already freed by free(), the behavior is UNDEFINED

If the pointer ptr is NULL, nothing happens

No **return** value

The C programmer is responsible **for** setting free the memory, when it is no more needed

Dynamic de-allocation: free(...)

Syntax:

void free(void *pointer);

Function

free() frees the memory block pointed out by the pointer, which has been reserved by a call, such as malloc(), calloc() ou realloc()

If the pointer has not been obtained by one of these functions, or it has been already freed by free(), the behavior is UNDEFINED

If the pointer ptr is NULL, nothing happens

No return value

The C programmer is responsible for setting free the memory, when it is no more needed

#291

1.17 Backup: Dynamic allocation: Example

Original slide 292

Dynamic allocation: Example

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int* p;
    int i;
    int taille = 100;
    p = (int*)(malloc (taille * sizeof(int)));
    if (p == NULL)
    {
        printf("Allocation impossible:");
        exit(EXIT_FAILURE);
    }
    for (i=0; i<taille;i++){ p[i]=0; }
    free(p);
}
```

#292

1.18 Backup: Dynamic Memory Allocation

Original slide 293

- Kahoot time!
- Q1 - Q5

- Kahoot time!
- Q1 - Q5

#293

1.19 Backup: 1D Array with size elements

Original slide 294

```
type *arr = (type *)malloc(size * sizeof(type));
```

```
int *arr=(int *)malloc(4*sizeof(int));
```

- Heap
- xxx
- 0x1000
- xxx
- 0x1004
- xxx
- 0x1008
- xxx
- 0x100C
- xxx
- xxx
- 0x1014
- xxx
- 0x1018
- xxx
- xxx
- xxx

- 0x1028
- xxx
- xxx

1D Array with size elements

```
type *arr = (type *)malloc(size * sizeof(type));
```

```
int *arr=(int *)malloc(4*sizeof(int));
```

	- Heap
-0x1000	- xxx
-0x1004	- xxx
-0x1008	- xxx
-0x100C	- xxx
	- xxx
-0x1014	- xxx
-0x1018	- xxx
	- xxx
	- xxx
-0x1028	- xxx
	- xxx
	- xxx
	#294

1.20 Backup: 1D Array with size characters -> string

Original slide 295

```
char *arr=(char *)malloc(4*sizeof(char));
```

- Heap
- xxx
- 0x1000
- xxx
- 0x1004
- xxx
- 0x1008
- xxx
- 0x100C
- xxx
- xxx
- 0x1014
- xxx
- 0x1018
- xxx
- xxx

- XXX
- 0x1028
- XXX
- XXX

1D Array with size characters -> string

```
char *arr=(char *)malloc(4*sizeof(char));
```

	- Heap
-0x1000	- XXX
-0x1004	- XXX
-0x1008	- XXX
-0x100C	- XXX
	- XXX
-0x1014	- XXX
-0x1018	- XXX
	- XXX
	- XXX
-0x1028	- XXX
	- XXX
	- XXX

#295

1.21 Backup: String declaration: Remarks

Original slide 296

Dynamic declaration

```
char* s=(char*)malloc(sizeof(char) * SIZE);
```

```
s = "hello";
```

We can assign a **char** pointer to another **char** pointer

```
char *p; p=s;
```

We can assign another set of characters

```
s="bye";
```

String declaration: Remarks

```
Dynamic declaration
char * s=(char *)malloc(sizeof(char) * SIZE);
s = "hello";
We can assign a char pointer to another char pointer
char * p; p=s;
We can assign another set of characters
s="bye";
```

#296

1.22 Backup: 2D Array: Single pointer

Original slide 297

```
type *arr= (type *)malloc(rows * cols * sizeof(type));
```

```
int *arr=(int *)malloc(4*3*sizeof(int));
```

- Heap
- xxx
- 0x1000
- xxx
- 0x1004
- xxx
- 0x1008
- xxx
- 0x100C
- xxx
- xxx
- 0x1014
- xxx
- 0x1018
- xxx
- xxx
- xxx

- 0x1028
- xxx
- xxx

2D Array: Single pointer

```
type *arr= (type *)malloc(rows * cols * sizeof(type));
```

int *arr=(int *)malloc(4*3*sizeof(int));

	- Heap
-0x1000	- xxx
-0x1004	- xxx
-0x1008	- xxx
-0x100C	- xxx
	- xxx
-0x1014	- xxx
-0x1018	- xxx
	- xxx
	- xxx
-0x1028	- xxx
	- xxx
	- xxx

#297

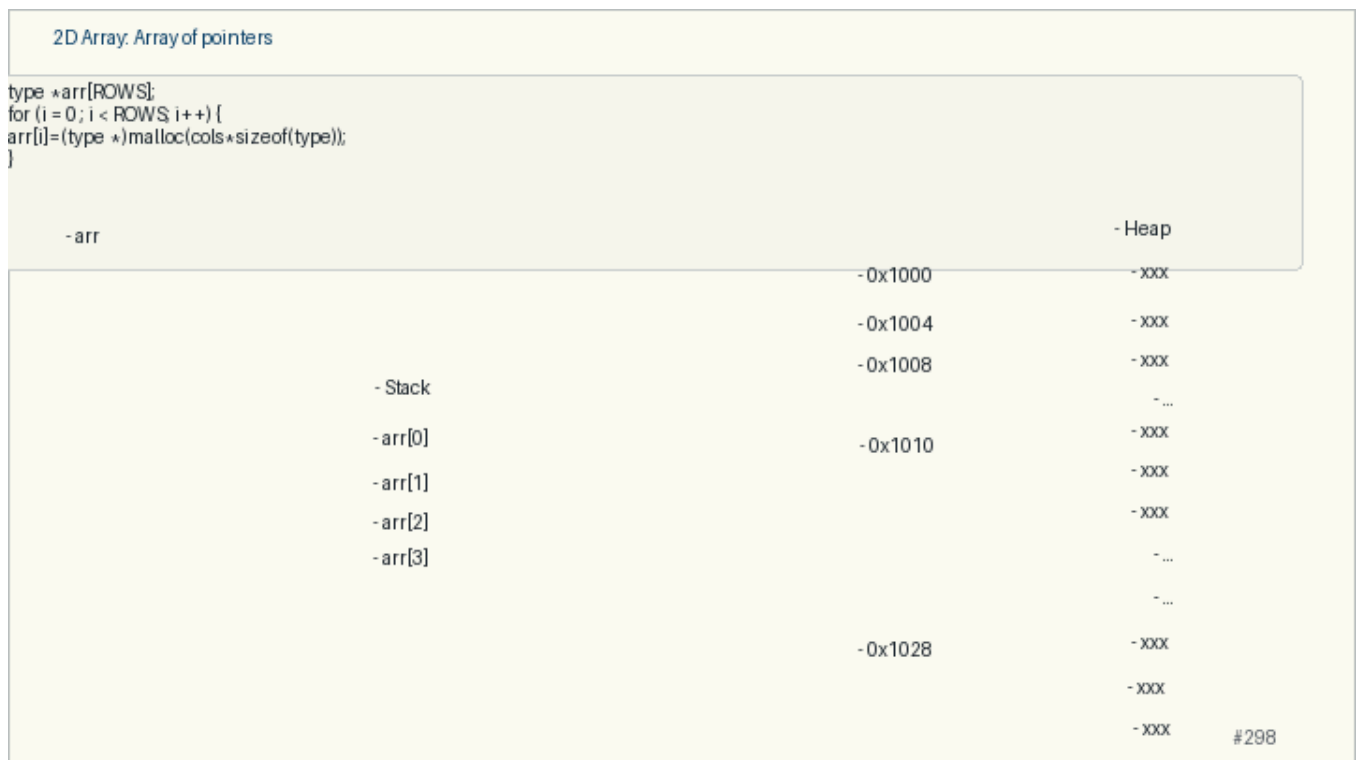
1.23 Backup: 2D Array: Array of pointers

Original slide 298

```
type *arr[ROWS];
for (i = 0 ; i < ROWS; i++) {
arr[i]=(type *)malloc(cols*sizeof(type));
}
```

- Heap
- arr
- xxx
- 0x1000
- xxx
- 0x1004
- xxx
- 0x1008
- Stack
- ...
- xxx
- arr[0]
- 0x1010
- xxx

- arr[1]
- xxx
- arr[2]
- ...
- arr[3]
- ...
- xxx
- 0x1028
- xxx
- xxx



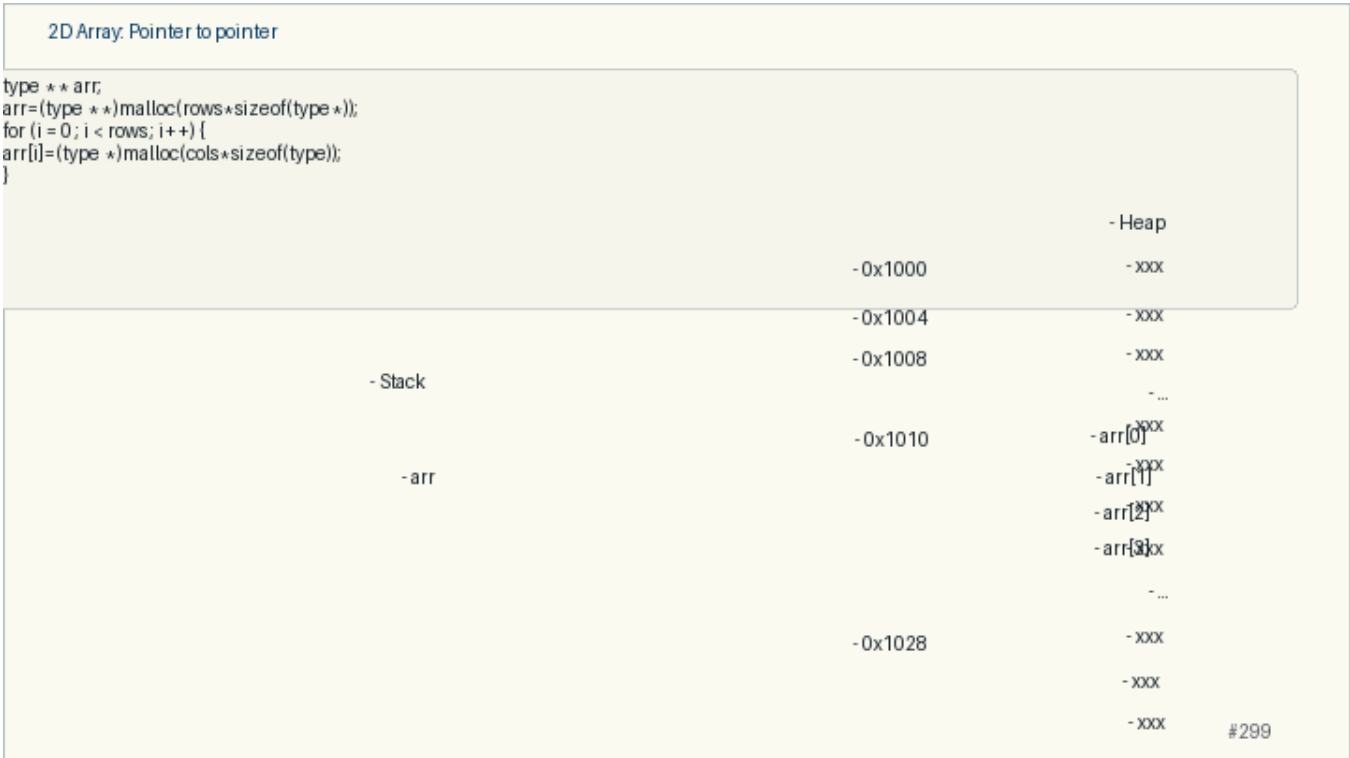
1.24 Backup: 2D Array: Pointer to pointer

Original slide 299

```
type ** arr;
arr=(type **)malloc(rows*sizeof(type*));
for (i = 0 ; i < rows ; i++) {
    arr[i]=(type *)malloc(cols*sizeof(type));
}
```

- Heap
- xxx
- 0x1000
- xxx
- 0x1004
- xxx

- 0x1008
- Stack
- ...
- xxx
- arr[0]
- 0x1010
- xxx
- arr[1]
- arr
- xxx
- arr[2]
- xxx
- arr[3]
- ...
- xxx
- 0x1028
- xxx
- xxx



1.25 Backup: Dynamic Memory Allocation

Original slide 300

- Kahoot time!
- Q6 - Q8

- Kahoot time!
- Q6 - Q8

#300

2 User-defined Data Types

Live pacing: about 8 min

Question. When stack objects are not enough, who owns memory and structure?

Core claim. Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.

Target capability. Students can pair allocation/free, define structs, and reason about linked/hash structures.

2.1 Main Path

- Start from the question: When stack objects are not enough, who owns memory and structure?
- Build the model: Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.
- End with a checkable action: Students can pair allocation/free, define structs, and reason about linked/hash structures.
- Keep only this slide on the horizontal path during the live lecture.

User-defined data types: Enum, typedef, struct

#301

2.2 Worked Example

- Use this as the live trace: Allocate one node, link it into a list, then remove it and state exactly who frees it.
- Representative source slide: 302
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Enumerated type declaration & definition

- Syntax:
 - enum identifier [element1,...]
- Function:
 - The enumeration creates a data type that groups a set of elements that behave as integral constants
 - Declares a new enumeration type with the name enum identifier
 - A variable is defined as enum identifier, it can be assigned one of the elements
- Example:
 - enum day(mon, tue, wed, thu, fri, sat, sun);
 - mon is coded by 0, tue by 1, ..., sun par 6
 - This is a new type called enum day
 - enum day today;
 - today=wed;

#302

2.3 Anecdote Or Motivation

Every `malloc` creates a future obligation. Good C code makes that obligation visible.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

2.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Structures

- Kahoot time!
- Q9 - Q12

#312

2.5 Backup Index

Original slides 301-312

User-defined data types: Enum, typedef, struct

#301

slide 301

Enumerated type declaration & definition

- Syntax:
 - enum identifier {element1,...}
- Function:
 - The enumeration creates a data type that groups a set of elements that behave as integral constants
 - Declares a new enumeration type with the name enum identifier
 - A variable is defined as enum identifier, it can be assigned one of the elements
- Example:
 - enum day{mon, tue, wen, thu, fri, sat, sun};
 - mon is coded by 0, tue by 1, ..., sun par 6
 - This is a new type called enum day
 - enum day today;
 - today=wen;

#302

slide 302

Structure type declaration

Syntax

```
struct identifier {  
    member definition;  
    ...  
};
```

Function:

A structure creates a data type that groups items of possibly different types into a single type.

Declares a new structure type named struct identifier

The members of the structure can be integral, float, arrays, pointers.

Example:

```
struct address{  
    char street[100];  
    int number;  
};
```

- NO space is reserved!
- It only creates a new type that describes the members of the group!

#303

slide 303

Structure type declaration

- The structure type declaration CANNOT be initialized!
- This is a new data type declaration, not a structure variable definition!
- Only when the structure variable is defined !
- The corresponding memory is reserved
- Now we can initialize the members of the structure

#304

slide 304

Structure variable definition & initialisation

Separately from the structure declaration:
`struct address MyAddress = {"Paul Bert", 12};`
During structure declaration:
`struct address{
 char street[100];
 int number;
} MyAddress = {"Paul Bert", 12};`

- Space is reserved!
- Equal to the one required to store the structure members

#305

slide 305

Structure padding

- Architecture of a computer processor is made in such a way so it can read 1 word from memory at a time
- e.g. 4 byte in 32 bit processor
- To use this processor advantage, the data are aligned in memory in words
- e.g. multiples of 4 bytes in 32-bit processor
- One or more memory addresses may LEFT EMPTY to achieve this alignment !

#306

slide 306

Anchors: User-defined data types: Enum, typedef, struct; Enumerated type declaration & definition; Structure type declaration; Structure variable definition & initialisation; Structure padding; Structure padding: Example

2.6 Backup: User-defined data types: Enum, typedef, struct

Original slide 301

User-defined data types: Enum, typedef, struct

#301

2.7 Backup: Enumerated type declaration & definition

Original slide 302

- Syntax:
- enum identifier {element1,...}

- Function:
- The enumeration creates a data type that groups a set of elements that behave as integral constants
- Declares a new enumeration type with the name enum identifier
- A variable is defined as enum identifier, it can be assigned one of the elements
- Example:
- `enum day{mon, tue, wen, thu, fri, sat, sun} ;`
- mon is coded by 0, tue by 1, ..., sun par 6
- This is a new type called enum day
- `enum day today;`
- `today=wen;`

Enumerated type declaration & definition

```
- Syntax:
- enum identifier {element1,...}
- Function:
- The enumeration creates a data type that groups a set of elements that behave as integral constants
- Declares a new enumeration type with the name enum identifier
- A variable is defined as enum identifier, it can be assigned one of the elements
- Example:
- enum day{mon, tue, wen, thu, fri, sat, sun};
- mon is coded by 0, tue by 1, ..., sun par 6
- This is a new type called enum day
- enum day today;
- today=wen;
```

#302

2.8 Backup: Structure type declaration

Original slide 303

Syntax

```
struct identifier {
member definition;
...
};
```

Function:

A structure creates a data type that groups items of possibly different types into a single type.

Declares a new structure type named **struct** identifier

The members of the structure can be integral, **float**, arrays, pointers.

Example:

```
struct address{
char street[100];
int number;
};
```

- NO space is reserved!
- It only creates a new type that describes the members of the group !

Structure type declaration

Syntax

```
struct identifier {  
    member definition;  
    ...  
};
```

Function:

A structure creates a data type that groups items of possibly different types into a single type.

Declares a new structure type named struct identifier

The members of the structure can be integral, float, arrays, pointers.

Example:

```
struct address{  
    char street[100];  
    int number;  
};
```

- NO space is reserved!

- It only creates a new type that describes the members of the group !

#303

2.9 Backup: Structure type declaration

Original slide 304

- The structure type declaration CANNOT be initialized!
- This is a new data type declaration, not a structure variable definition!
- Only when the structure variable is defined !
- The corresponding memory is reserved
- Now we can initialize the members of the structure

Structure type declaration

- The structure type declaration CANNOT be initialized!
- This is a new data type declaration, not a structure variable definition!
- Only when the structure variable is defined !
- The corresponding memory is reserved
- Now we can initialize the members of the structure

#304

2.10 Backup: Structure variable definition & initialisation

Original slide 305

```
Separately from the structure declaration:  
struct address MyAddress = {"Paul Bert", 12};  
During structure declaration:  
struct address{  
char street[100];  
int number;  
} MyAddress = {"Paul Bert", 12};
```

- Space is reserved!
- Equal to the one required to store the structure members aligned

Structure variable definition & initialisation

```
Separately from the structure declaration:  
struct address MyAddress = {"Paul Bert", 12};  
During structure declaration:  
struct address{  
char street[100];  
int number;  
} MyAddress = {"Paul Bert", 12};
```

- Space is reserved!
- Equal to the one required to store the structure members aligned

#305

2.11 Backup: Structure padding

Original slide 306

- Architecture of a computer processor is made in such a way so it can read 1 word from memory at a time
- e.g. 4 byte in 32 bit processor
- To use this processor advantage, the data are aligned in memory in words
- e.g. multiples of 4 bytes in 32-bit processor
- One or more memory addresses may LEFT EMPTY to achieve this alignment !

Structure padding

- Architecture of a computer processor is made in such a way so it can read 1 word from memory at a time
- e.g. 4 byte in 32 bit processor
- To use this processor advantage, the data are aligned in memory in words
- e.g. multiples of 4 bytes in 32-bit processor
- One or more memory addresses may LEFT EMPTY to achieve this alignment!

#306

2.12 Backup: Structure padding: Example

Original slide 307

```
struct structure1
{
    int id1;
    int id2;
    char c1;
    char c2;
    float ratio;
};
```

```
struct structure1
{
    int id1;
    char c1;
    int id2;
    char c2;
    float ratio;
};
```

- 0x1000
- 4 bytes
- 0x1000
- id1
- id1
- 0x1004
- 0x1004

- 4 bytes
- id2
- c1
- 0x1008
- 0x1008
- 1 byte
- 1 byte
- id2
- c1
- c2
- rat
- 0x100C
- 0x100C
- 4 bytes
- c2
- io
- ratio
- 0x1010
- 0x1010
- ratio

Structure padding: Example

```
struct structure1
{
    int id1;
    int id2;
    char c1;
    char c2;
    float ratio;
};
```

```
struct structure1
{
    int id1;
    char c1;
    int id2;
    char c2;
    float ratio;
};
```

-0x1000	- id1	- 4 bytes		-0x1000	- id1
-0x1004	- id2	- 4 bytes		-0x1004	- c1
-0x1008	- c1 - c2 - rat	- 1 byte - 1 byte		-0x1008	- id2
-0x100C	- ratio	- 4 bytes		-0x100C	- c2
-0x1010				-0x1010	- ratio

#307

2.13 Backup: Structure variable: Access members

Original slide 308

```

struct address{
    char street[100];
    int number;
};

```

Through a structure variable: .

```

struct address MyAddress;
strcpy(MyAddress.street, "Paul Bert");
MyAddress.number=12;

```

Through a pointer to the structure variable: ->

```

struct address MyAddress;
struct address *MyAddressPtr = &MyAddress;
strcpy(MyAddressPtr->street, "Paul Bert");
MyAddressPtr->number=12;

```

Structure variable: Access members

```

structaddress{
    char street[100];
    int number;
};

```

Through a structure variable: .

```

structaddress MyAddress;
strcpy(MyAddress.street, "Paul Bert");
MyAddress.number=12;

```

Through a pointer to the structure variable: ->

```

structaddress MyAddress;
structaddress *MyAddressPtr = &MyAddress;
strcpy(MyAddressPtr->street, "Paul Bert");
MyAddressPtr->number=12;

```

#308

2.14 Backup: Definition of a new type

Original slide 309

Syntax

```

typedef type IDENTIFIER;

```

Examples

```

typedef int LENGHT;

```

```

typedef enum {false, true} BOOLEAN ;

```

```

typedef struct date date;

```

Use of **typedef** to avoid repetition of keyword **struct**

Remark

There is no identifier **for** the **enum** in the definition of the type BOOLEAN

Definition of a new type

Syntax
typedef type IDENTIFIER;
Examples

#309

2.15 Backup: Structure type: Examples

Original slide 310

Structure type: Examples

```
struct date /* déclaration du type struct date */
{
    short j ;
    short m ;
    int a ;
} ;
```

```
/* déclaration du type struct UNE_DATE */
typedef enum {lun, mar, mer, jeu, ven, sam, dim} JOUR ;
typedef struct
{
    JOUR j ;
    struct date d ;
} UNE_DATE ;
```

#310

2.16 Backup: Access struct members

Original slide 311

- Variable structure UNE_DATE foo = { mer, {3 ,9 ,2014 } } ; foo.j = mer ; foo.d.a = 2014 ;
- Structure pointer UNE_DATE* queljour = &foo; queljour->j = foo.j;
- queljour->d.a = 2014;

Access struct members

```
- Variable structure UNE_DATE foo = { mer, {3,9,2014}}; foo.j = mer; foo.d.a = 2014;  
- Structure pointer UNE_DATE* queljour = &foo; queljour->j = foo.j;  
- queljour->d.a = 2014;
```

#311

2.17 Backup: Structures

Original slide 312

- Kahoot time!
- Q9 - Q12

Structures

```
- Kahoot time!  
- Q9 - Q12
```

#312

3 Lists

Live pacing: about 16 min

Question. When stack objects are not enough, who owns memory and structure?

Core claim. Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.

Target capability. Students can pair allocation/free, define structs, and reason about linked/hash structures.

3.1 Main Path

- Start from the question: When stack objects are not enough, who owns memory and structure?
- Build the model: Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.
- End with a checkable action: Students can pair allocation/free, define structs, and reason about linked/hash structures.
- Keep only this slide on the horizontal path during the live lecture.

- Data structures: Lists and Hash tables

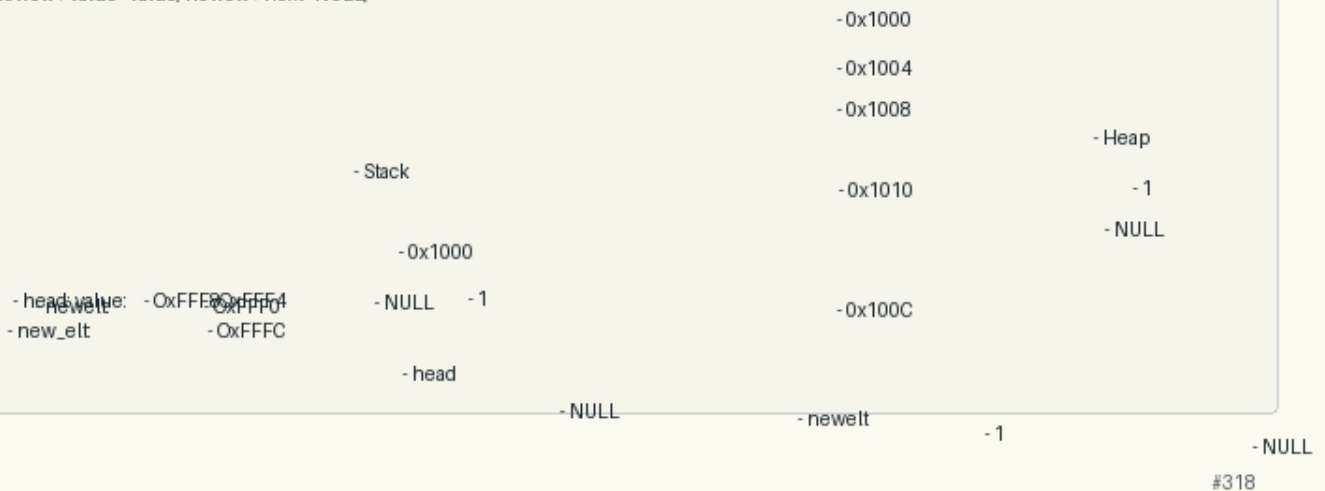
#313

3.2 Worked Example

- Use this as the live trace: Allocate one node, link it into a list, then remove it and state exactly who frees it.
- Representative source slide: 318
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Linked list Example

```
int main(){
struct list_elem_t * head = NULL;
struct list_elem_t * new_elt = create_element(1);
...
}
list_elem_t* create_element(int value){
list_elem_t* newelt = malloc(sizeof(struct list_elem_t));
newelt->value=value; newelt->next=NULL;
}
```



3.3 Anecdote Or Motivation

Every `malloc` creates a future obligation. Good C code makes that obligation visible.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

3.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Doubly linked list

Each element has

- data
- prev: Link to the previous element
- next: Link to the next element

Advantages

- Traversed both in forward and backward direction
- Delete operation is more efficient, previous pointer is given

Singly list: We have to traverse the list to find it

Disadvantages:

- Extra memory space for storing the previous pointer
- All operations have to maintain this extra pointer.

#355

3.5 Backup Index

Original slides 313-355

- Data structures: Lists and Hash tables

#313

slide 313

Linked lists Vs arrays

- Linear data structure (like arrays)
- Linked list elements NOT stored in contiguous location
- are linked using pointers

Head - element - element - element - element - Tail - NULL

#314

slide 314

Why Linked list?

- Array limitations:
- Fixed size: We must know the upper limit on the number of elements in advance
- Insert/Delete an element: Expensive
- Create room by shifting existing elements !
- Ex. : Add new element 101 in an array with sorted ids
- id[]=[100, 103, 105, 200]
- List Advantages:
- Dynamic size
- Ease of insertion/deletion
- List Drawbacks:
- Random access is not allowed
- Access elements sequentially

#315

slide 315

Linked list

- Each element has
- data
- next: Link to the next element
- In C, such an element can be represented using structures

- element

- next

Head

- element

- element

- element

- data

- element

- Tail

- NULL



#316

slide 316

Linked list Example

```
int main(){
struct list_elem_t * head = NULL;
...
}
```

```
struct list_elem_t{
int value;
list_elem_t * next;
}
```

- Stack

- head: - 0xFFFF8 - NULL

- head - NULL

#317

slide 317

Linked list Example

```
int main(){
struct list_elem_t * head = NULL;
struct list_elem_t * new_elt = create_element(1);
...
}
list_elem_t* create_element(int value){
list_elem_t* newelt = malloc(sizeof(struct list_elem_t));
newelt->value=value; newelt->next=NULL;
}
```

- Stack

- head: - 0xFFFF8 - 0xFFFF4 - 1 - NULL

- new_elt - 0xFFFFC - NULL

- head - 0xFFFFC - 0x1000 - 1 - NULL

- newelt - 0x100C - NULL

- Heap

- 0x1000

- 0x1004

- 0x1008

- 0x1010 - 1 - NULL

#318

slide 318

Anchors: Data structures: Lists and Hash tables; Linked lists Vs arrays; Why Linked list?; Linked list; Linked list: Example; Why pass by reference?

3.6 Backup: Data structures: Lists and Hash tables

Original slide 313

- Data structures: Lists and Hash tables

#313

3.7 Backup: Linked lists Vs arrays

Original slide 314

- Linear data structure (like arrays)
- Linked list elements NOT stored in contiguous location
- are linked using pointers
- element
- element
- element
- element
- Head
- Tail
- NULL

Linked lists Vs arrays

- Linear data structure (like arrays)
- Linked list elements NOT stored in contiguous location
- are linked using pointers



#314

3.8 Backup: Why Linked list?

Original slide 315

- Array limitations:
- Fixed size: We must know the upper limit on the number of elements in advance
- Insert/Delete an element: Expensive
- Create room by shifting existing elements !
- Ex. : Add new element 101 in an array with sorted ids
- `id[]=[100, 103, 105, 200]`
- List Advantages:
- Dynamic size
- Ease of insertion/deletion
- List Drawbacks:
- Random access is not allowed
- Access elements sequentially
- Extra memory space required for the pointer to each element

Why Linked list?

- Array limitations:
- Fixed size: We must know the upper limit on the number of elements in advance
- Insert/Delete an element: Expensive
- Create room by shifting existing elements !
- Ex. : Add new element 101 in an array with sorted ids
- id[]=[100, 103, 105, 200]
- List Advantages:
- Dynamic size
- Ease of insertion/deletion
- List Drawbacks:
- Random access is not allowed
- Access elements sequentially

#315

3.9 Backup: Linked list

Original slide 316

- Each element has
- data
- next: Link to the next element
- In C, such an element can be represented using structures
- element
- next
- data
- element
- element
- element
- element
- Head
- Tail
- NULL



3.10 Backup: Linked list: Example

Original slide 317

```
int main(){
  struct list_elem_t *head = NULL;
  ...
}
```

```
struct list_elem_t {
  int value;
  list_elem_t *next;
};
```

- Stack
- head:
- 0xFFFF8
- NULL
- head
- NULL

Linked list Example

```
int main(){
struct list_elem_t *head = NULL;
...
}
```

```
struct list_elem_t{
int value;
list_elem_t *next;
};
```

- Stack

- head: - 0xFFFF8 - NULL

- head

- NULL

#317

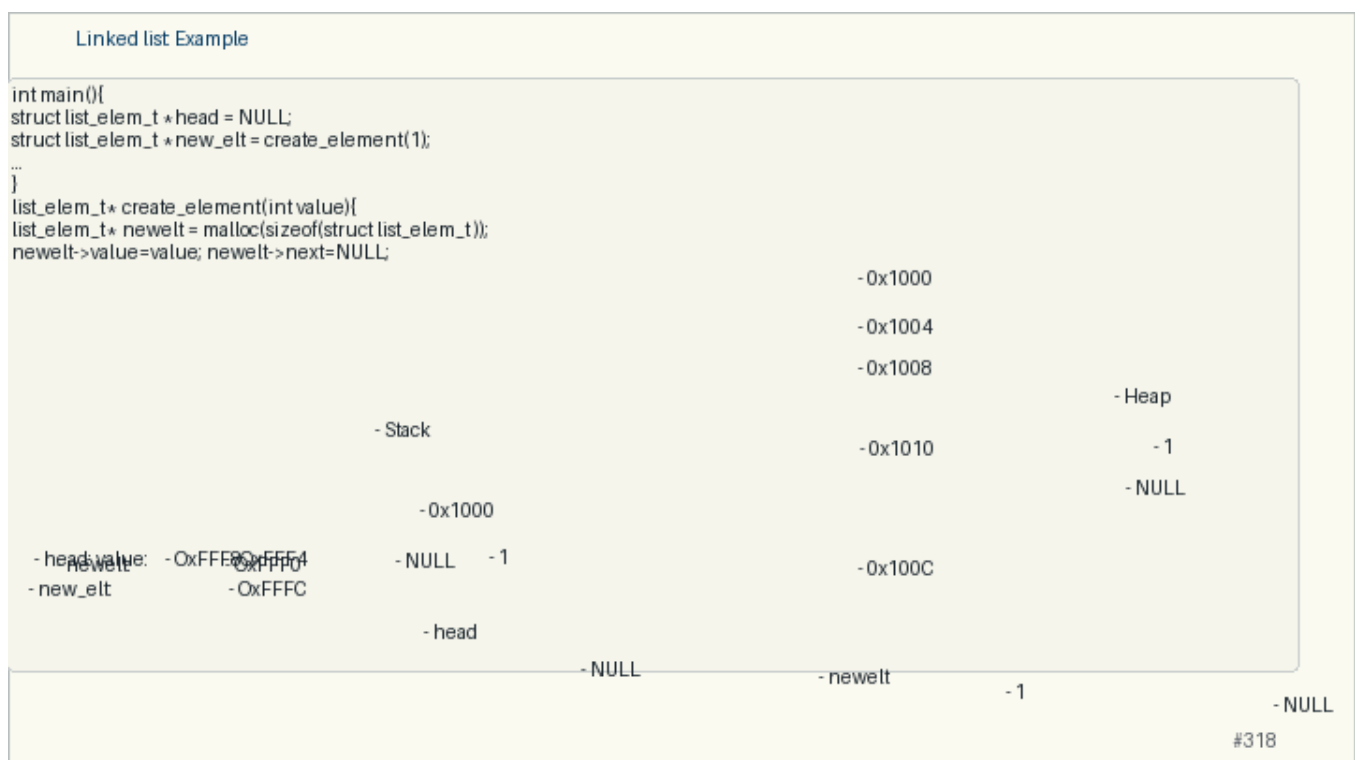
3.11 Backup: Linked list: Example

Original slide 318

```
int main(){
struct list_elem_t *head = NULL;
struct list_elem_t *new_elt = create_element(1);
...
}
list_elem_t* create_element(int value){
list_elem_t* newelt = malloc(sizeof(struct list_elem_t ));
newelt->value=value; newelt->next=NULL;
```

- 0x1000
- 0x1004
- 0x1008
- Heap
- Stack
- 1
- 0x1010
- NULL
- 0x1000
- 1
- value:
- 0xFFFF4
- head:
- 0xFFFF8

- NULL
- newelt:
- 0xFFFF0
- 0x100C
- new_elt:
- 0xFFFC
- head
- NULL
- newelt
- 1
- NULL



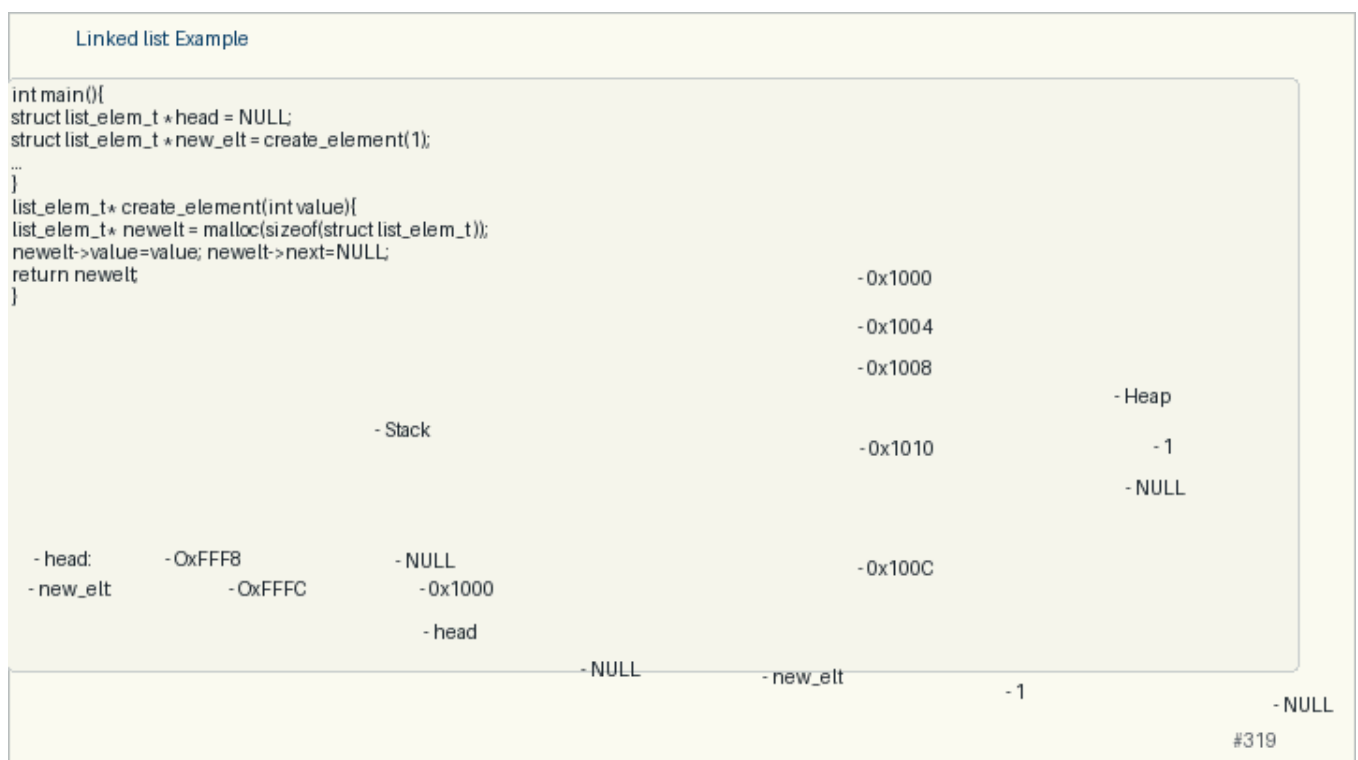
3.12 Backup: Linked list: Example

Original slide 319

```
int main(){
  struct list_elem_t *head = NULL;
  struct list_elem_t *new_elt = create_element(1);
  ...
}
list_elem_t* create_element(int value){
  list_elem_t* newelt = malloc(sizeof(struct list_elem_t ));
  newelt->value=value; newelt->next=NULL;
  return newelt;
}
```

- 0x1000
- 0x1004

- 0x1008
- Heap
- Stack
- 1
- 0x1010
- NULL
- head:
- 0xFF8
- NULL
- 0x100C
- new_elt:
- 0xFFC
- 0x1000
- head
- NULL
- new_elt
- 1
- NULL



3.13 Backup: Linked list: Example

Original slide 320

```

int main(){
  struct list_elem_t *head = NULL;

```

```

struct list_elem_t *new_elt = create_element(1);
head = new_elt;
}

```

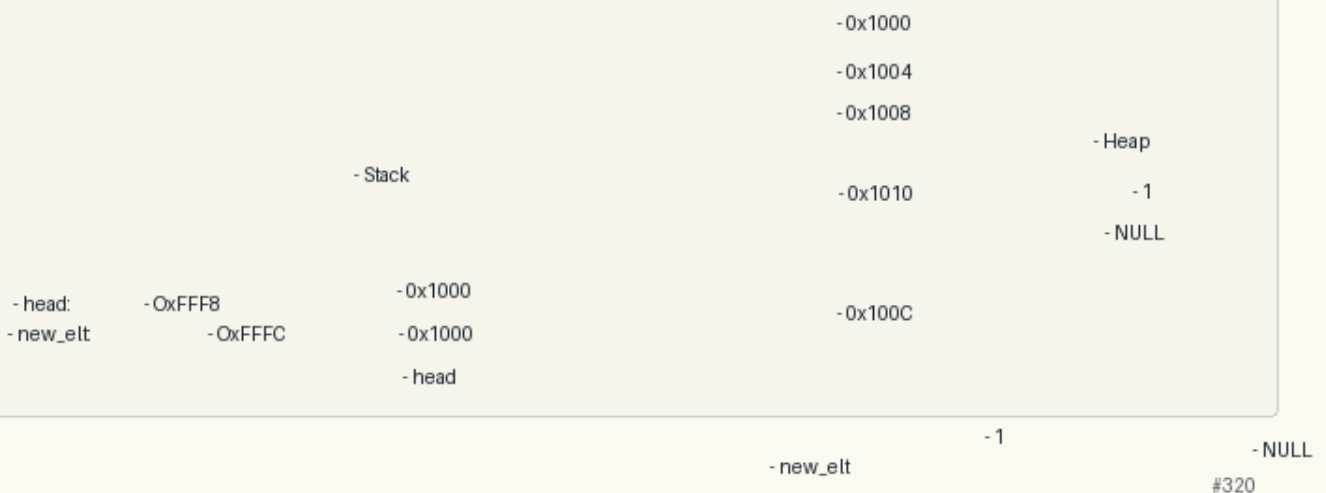
- 0x1000
- 0x1004
- 0x1008
- Heap
- Stack
- 1
- 0x1010
- NULL
- 0x1000
- head:
- 0xFFF8
- 0x100C
- new_elt:
- 0xFFFC
- 0x1000
- head
- 1
- NULL
- new_elt

Linked list Example

```

int main(){
struct list_elem_t *head = NULL;
struct list_elem_t *new_elt = create_element(1);
head = new_elt;
}

```



3.14 Backup: Linked list: Example

Original slide 321

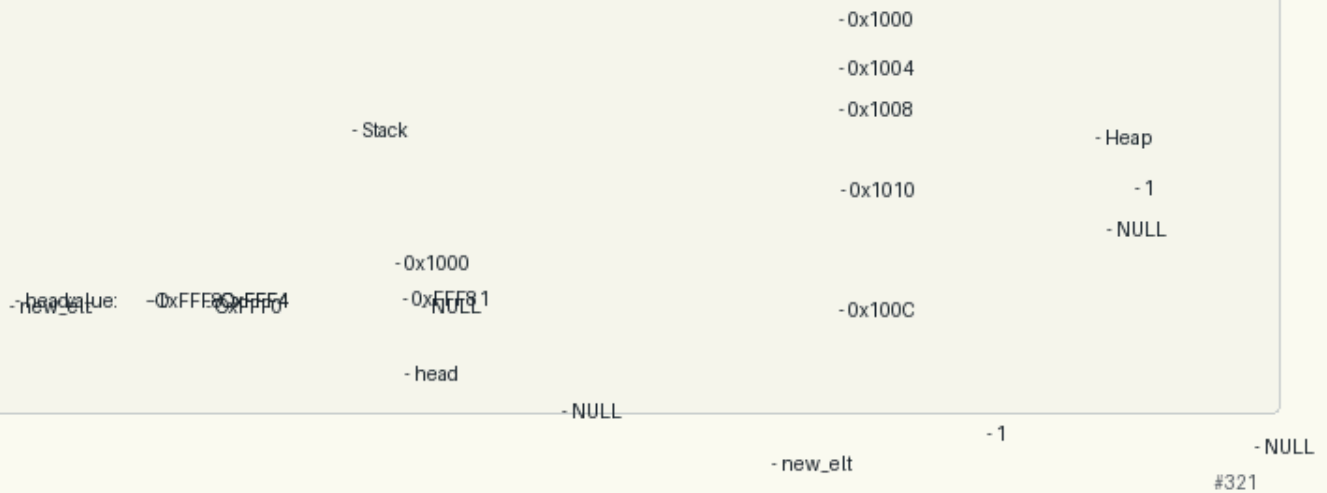
```
int main(){
    struct list_elem_t *head = NULL;
    insert_head(&head, 1);
}

int insert_head (list_elem_t * * l, int value){
    list_elem_t * new_elt = create_element (value);
    new_elt->next = *l;
}
```

- 0x1000
- 0x1004
- 0x1008
- Stack
- Heap
- 1
- 0x1010
- NULL
- 0x1000
- 0xFFFF8
- 1
- value:
- l:
- 0xFFFF4
- 0xFFFF4
- head:
- 0xFFFF8
- new_elt:
- 0xFFFF0
- NULL
- 0x100C
- head
- NULL
- 1
- NULL
- new_elt

Linked list Example

```
int main(){
struct list_elem_t * head = NULL;
insert_head(&head,1);
}
int insert_head (list_elem_t * * l, int value){
list_elem_t * new_elt = create_element (value);
new_elt->next = *l;
```



3.15 Backup: Linked list: Example

Original slide 322

```
int main(){
struct list_elem_t *head = NULL;
insert_head(&head,1);
}
int insert_head (list_elem_t * * l, int value){
list_elem_t * new_elt = create_element (value);
new_elt->next = *l;
*l = new_elt;
```

- 0x1000
- 0x1004
- 0x1008
- Stack
- Heap
- 1
- 0x1010
- NULL
- 0x1000
- 0xFFFF8
- 1
- value:
- l:
- 0xFFFF4

- Linked list Example

```

int main(){
struct list_elem_t *head = NULL;
insert_head(&head,1);
}

int insert_head (list_elem_t * *l, int value){
list_elem_t * new_elt = create_element (value);
new_elt->next = *l;
*l = new_elt;
}

```

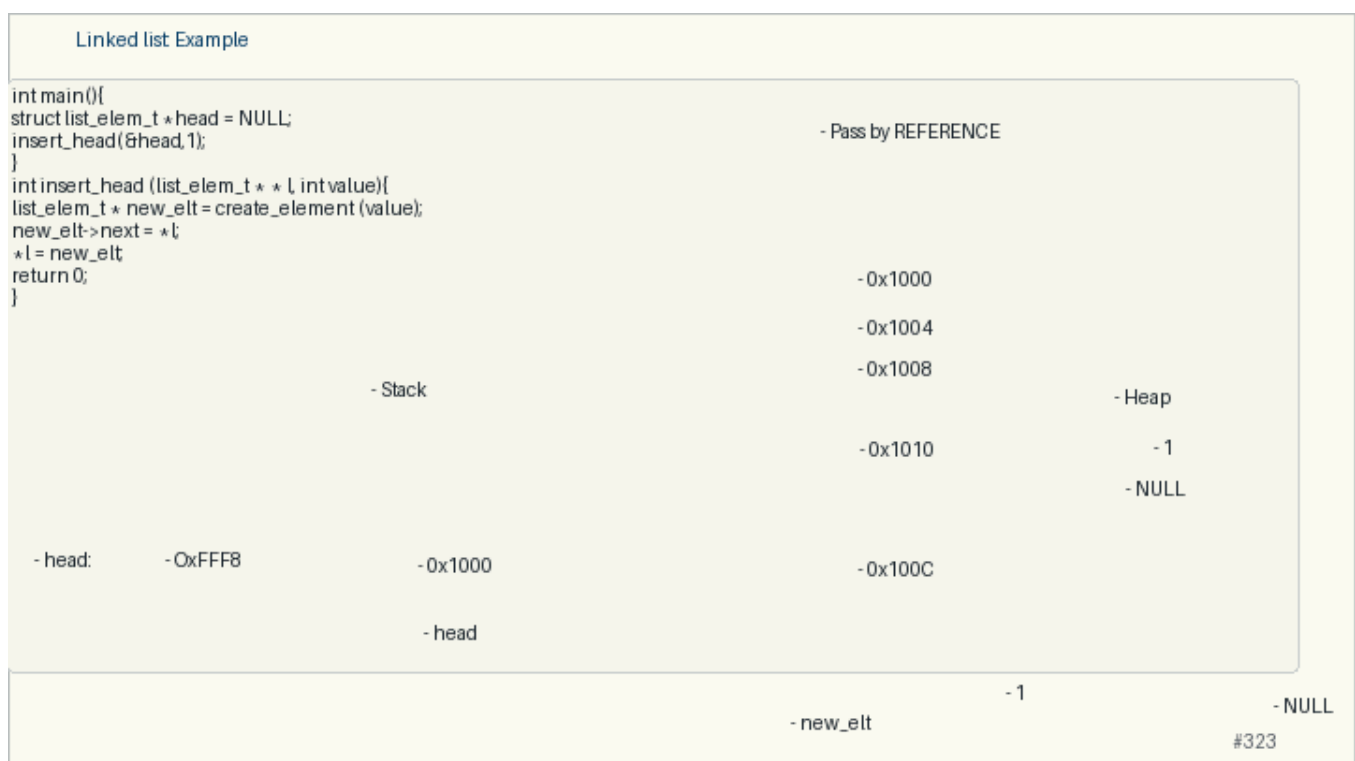
Memory diagram showing the state of the program:

 - Stack:**
 - `head` at `0x1000` points to `0xFFFF0000`.
 - `new_elt` at `0x1004` points to `0xFFFF0000`.
 - Heap:**
 - `new_elt` at `0x1008` points to `0x1010`.
 - `0x1010` points to `0x100C` (NULL).

Original slide 323

- Pass by REFERENCE

- 0x1000
- 0x1004
- 0x1008
- Stack
- Heap
- 1
- 0x1010
- NULL
- head:
- 0xFFFF8
- 0x1000
- 0x100C
- head
- 1
- NULL
- new_elt



3.17 Backup: Why pass by reference?

Original slide 324

```

int main(){
struct list_elem_t *head = NULL;
insert_head(head, 1);
}

```

```
int insert_head (list_elem_t * l, int value){
list_elem_t * new_elt = create_element (value);
```

- Pass by VALUE
- 0x1000
- 0x1004
- 0x1008
- Stack
- Heap
- 1
- 0x1010
- NULL
- 0x1000
- NULL
- 1
- value:
- l:
- 0xFFF4
- 0xFFF4
- head:
- 0xFFF8
- NULL
- new_elt:
- 0xFFF0
- 0x100C
- head
- 1
- NULL
- NULL
- 1
- NULL
- new_elt

Why pass by reference?

```

int main(){
    struct list_elem_t * head = NULL;
    insert_head(head, 1);
}

int insert_head (list_elem_t * l, int value){
    list_elem_t * new_elt = create_element (value);

```

The diagram illustrates the memory state during the execution of the provided C code. It shows two memory spaces: the Stack and the Heap.

- Stack:**
 - `head`: Points to address `0x1000` and contains the value `NULL`.
 - `new_elt`: Points to address `0x1004`. It contains:
 - Address `0x1008`: `0x1008`
 - Address `0x1010`: `1`
 - Address `0x1010`: `NULL`
 - Return value: Points to address `0x100C` and contains `NULL`.
- Heap:**
 - `new_elt`: Points to address `0xFFE0`. It contains:
 - Address `0xFFE4`: `0xFFE4`
 - Address `0xFFE8`: `0xFFE8`
 - Address `0xFFFC`: `0x1000`
 - Address `0x100C`: `0x100C`
 - Address `0x1010`: `1`
 - Address `0x1010`: `NULL`

3.18 Backup: Why pass by reference?

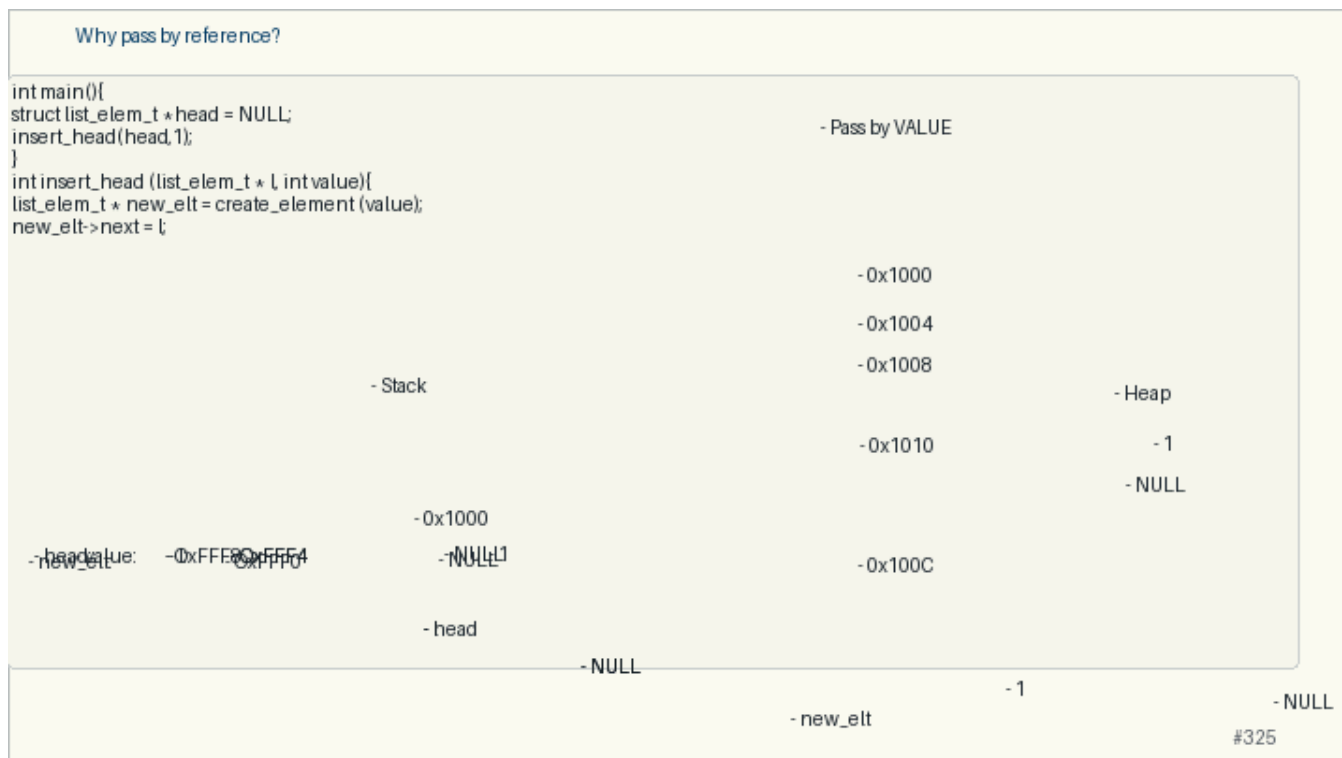
Original slide 325

```
int main(){
    struct list_elem_t *head = NULL;
    insert_head(head, 1);
}

int insert_head (list_elem_t * l, int value){
    list_elem_t * new_elt = create_element (value);
    new_elt->next = l;
}
```

- Pass by VALUE
- 0x1000
- 0x1004
- 0x1008
- Stack
- Heap
- 1
- 0x1010
- NULL
- 0x1000
- NULL
- 1
- value:
- l:
- 0xFFFF

- 0xFFFF4
- head:
- 0xFFFF8
- NULL
- new_elt:
- 0xFFFF0
- 0x100C
- head
- 1
- NULL
- NULL
- 1
- NULL
- new_elt



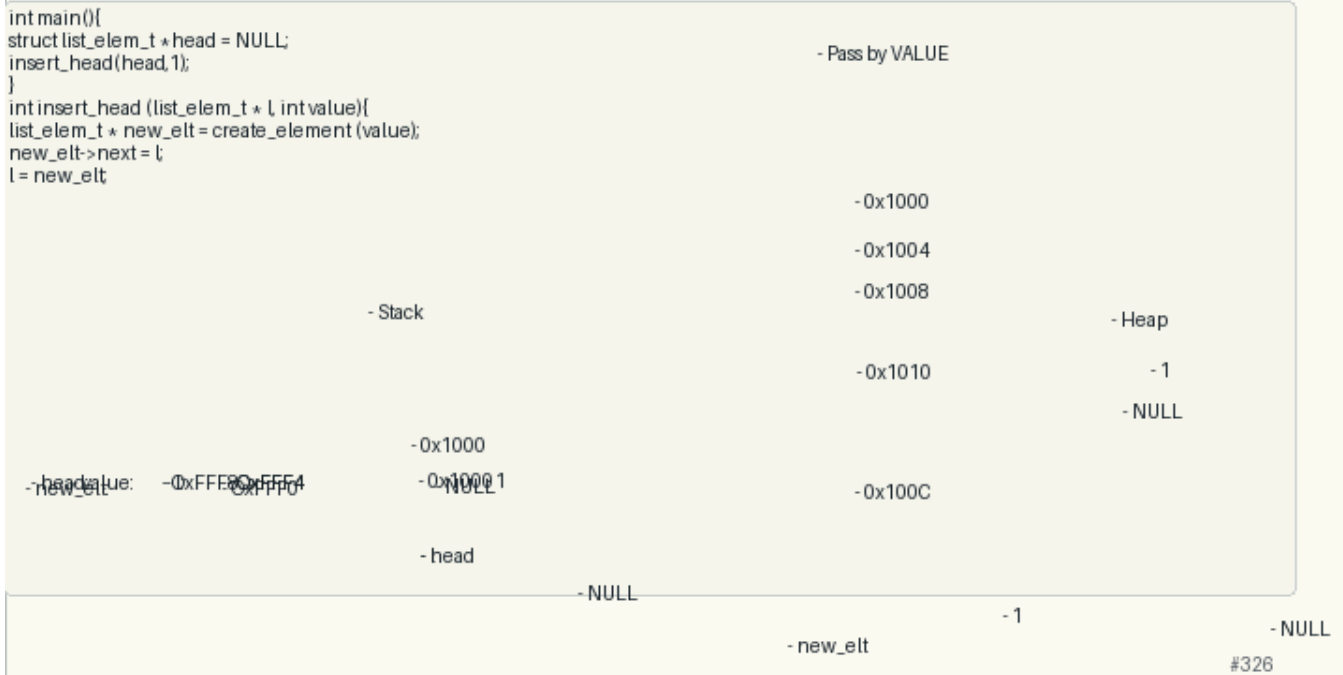
3.19 Backup: Why pass by reference?

Original slide 326

```
int main(){
    struct list_elem_t *head = NULL;
    insert_head(head, 1);
}
int insert_head (list_elem_t * l, int value){
    list_elem_t * new_elt = create_element (value);
    new_elt->next = l;
    l = new_elt;
}
```

- Pass by VALUE
- 0x1000
- 0x1004
- 0x1008
- Stack
- Heap
- 1
- 0x1010
- NULL
- 0x1000
- 0x1000
- 1
- value:
- l:
- 0xFFF4
- 0xFFF4
- head:
- 0xFFF8
- NULL
- new_elt:
- 0xFFF0
- 0x100C
- head
- 1
- NULL
- 1
- NULL
- new_elt

Why pass by reference?



3.20 Backup: Why pass by reference?

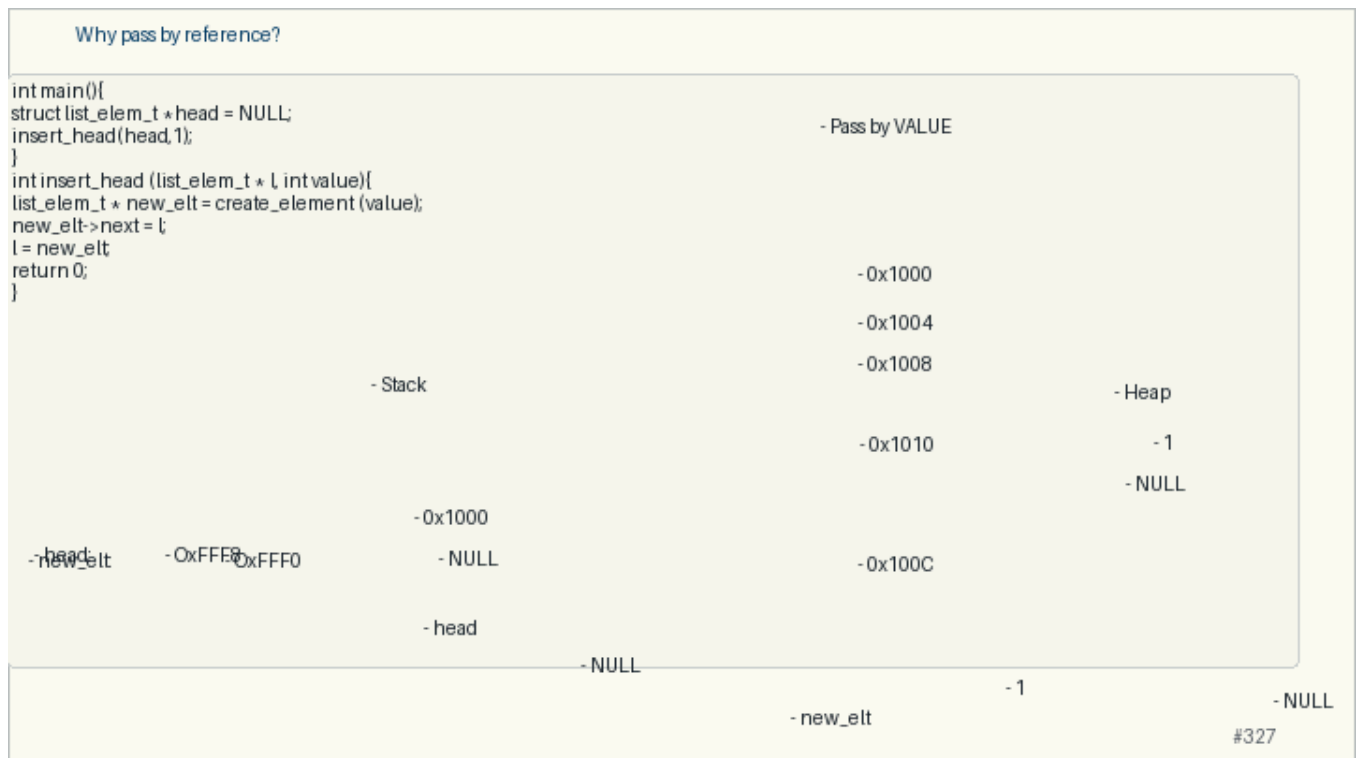
Original slide 327

```
int main(){
    struct list_elem_t *head = NULL;
    insert_head(head, 1);
}

int insert_head (list_elem_t * l, int value){
    list_elem_t * new_elt = create_element (value);
    new_elt->next = l;
    l = new_elt;
    return 0;
}
```

- Pass by VALUE
- 0x1000
- 0x1004
- 0x1008
- Stack
- Heap
- 1
- 0x1010
- NULL
- 0x1000
- head:
- 0x100C
- NULL

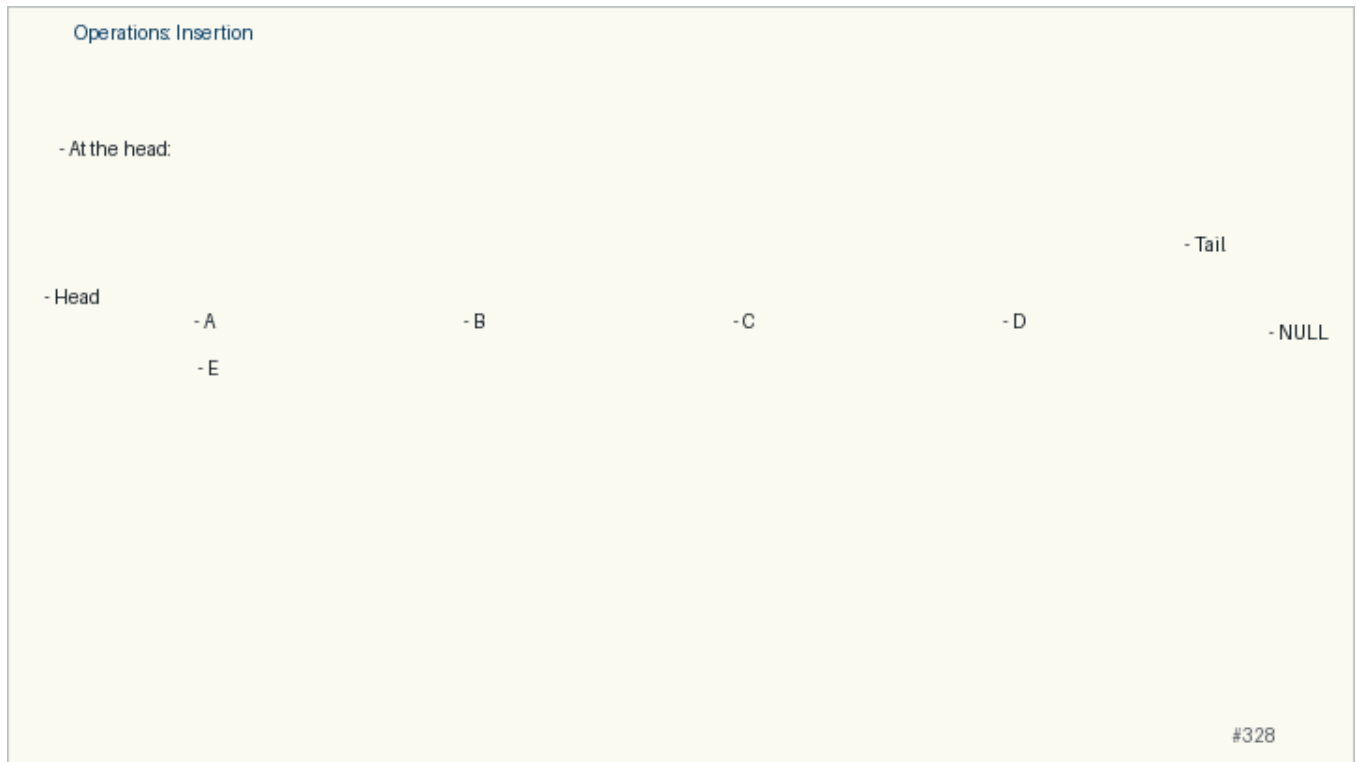
- new_elt:
- 0xFFFF0
- 0x100C
- head
- NULL
- 1
- NULL
- new_elt



3.21 Backup: Operations: Insertion

Original slide 328

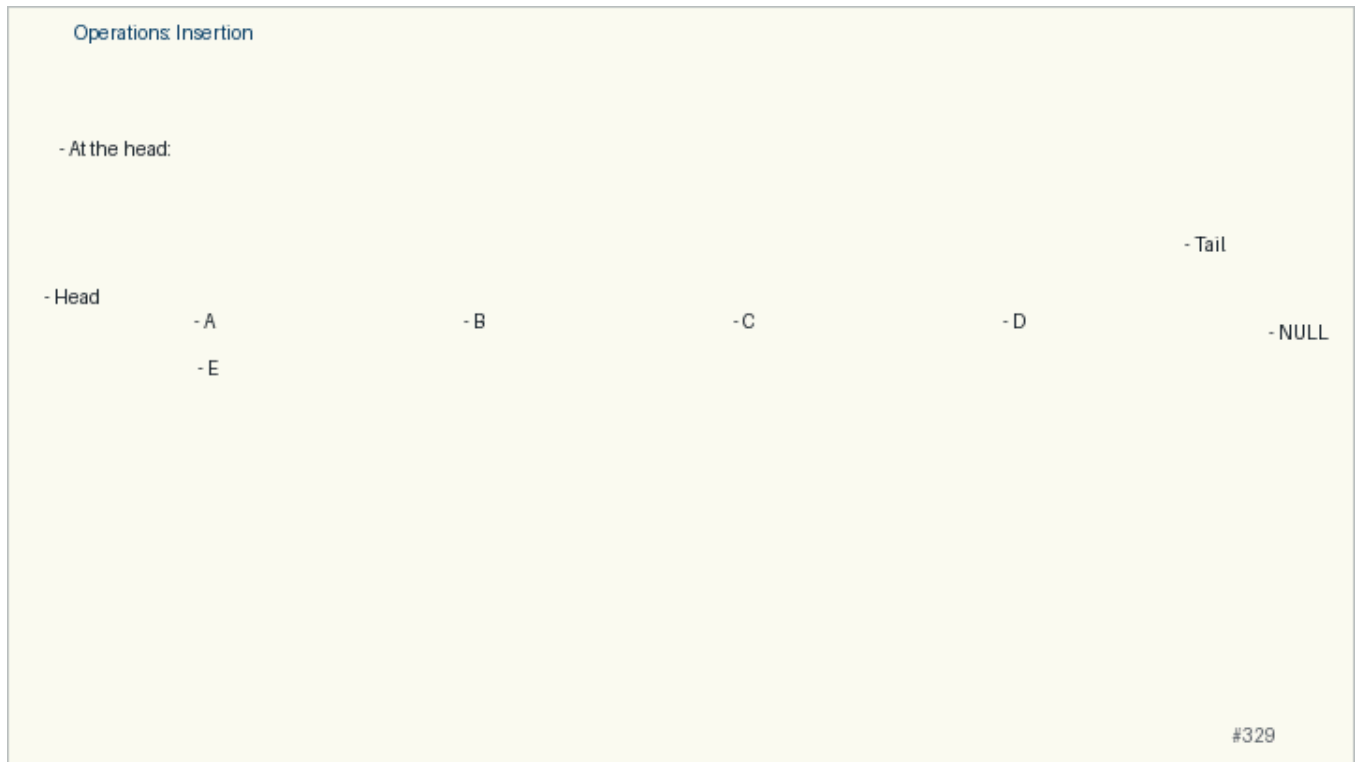
- At the head:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.22 Backup: Operations: Insertion

Original slide 329

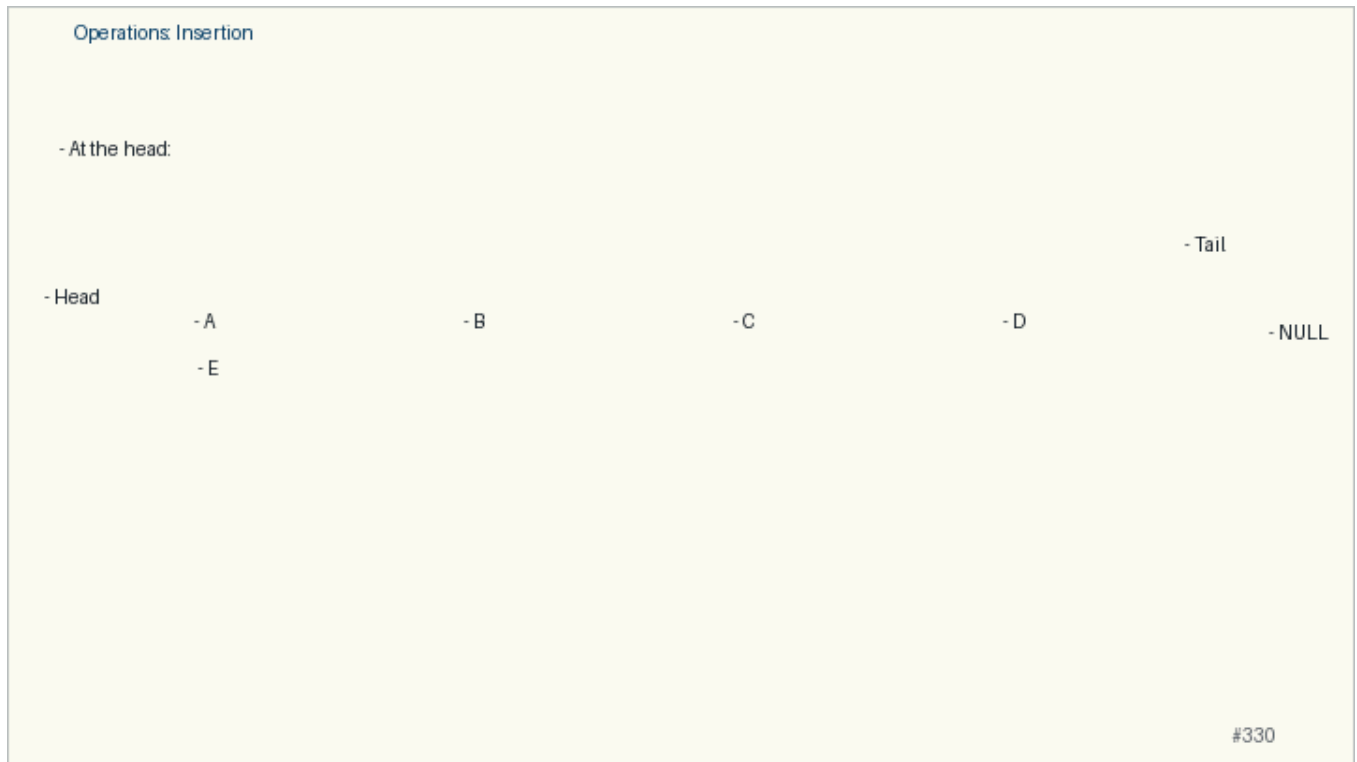
- At the head:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.23 Backup: Operations: Insertion

Original slide 330

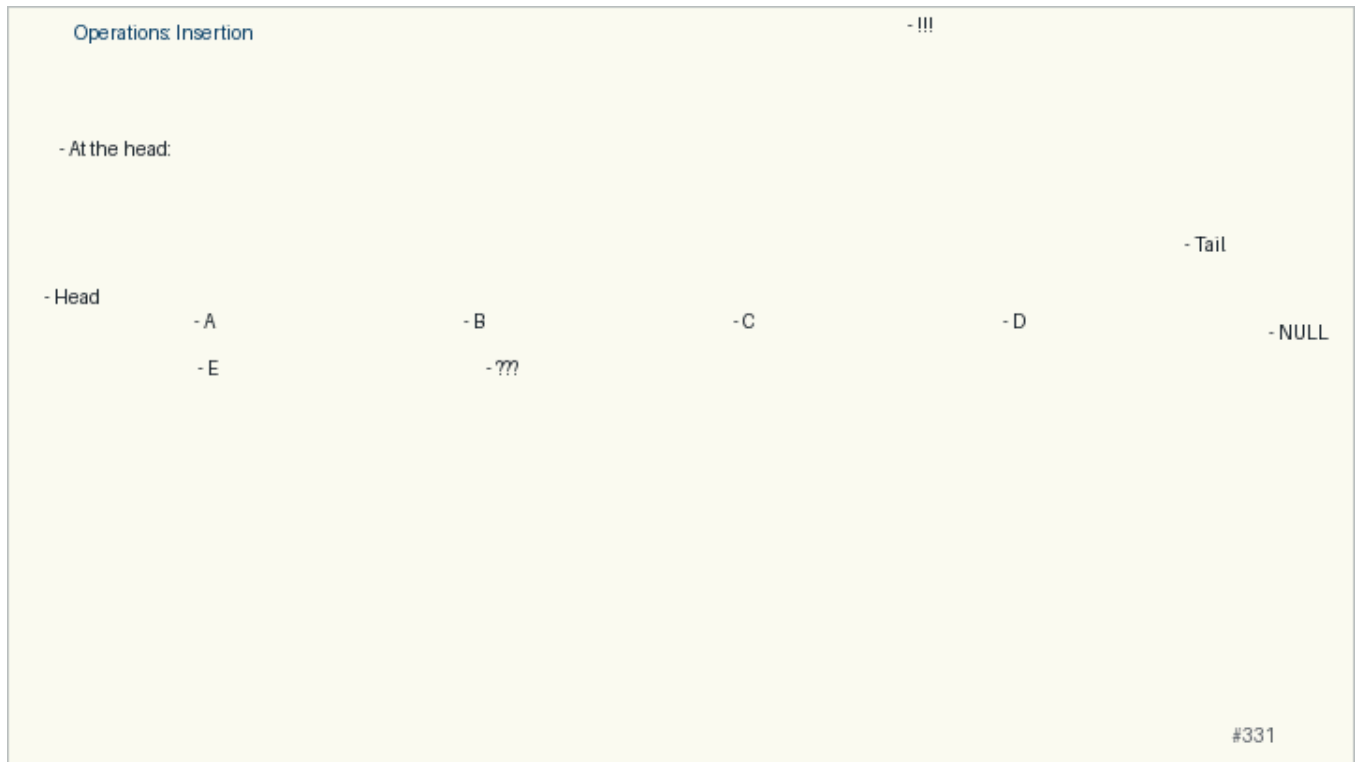
- At the head:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.24 Backup: Operations: Insertion

Original slide 331

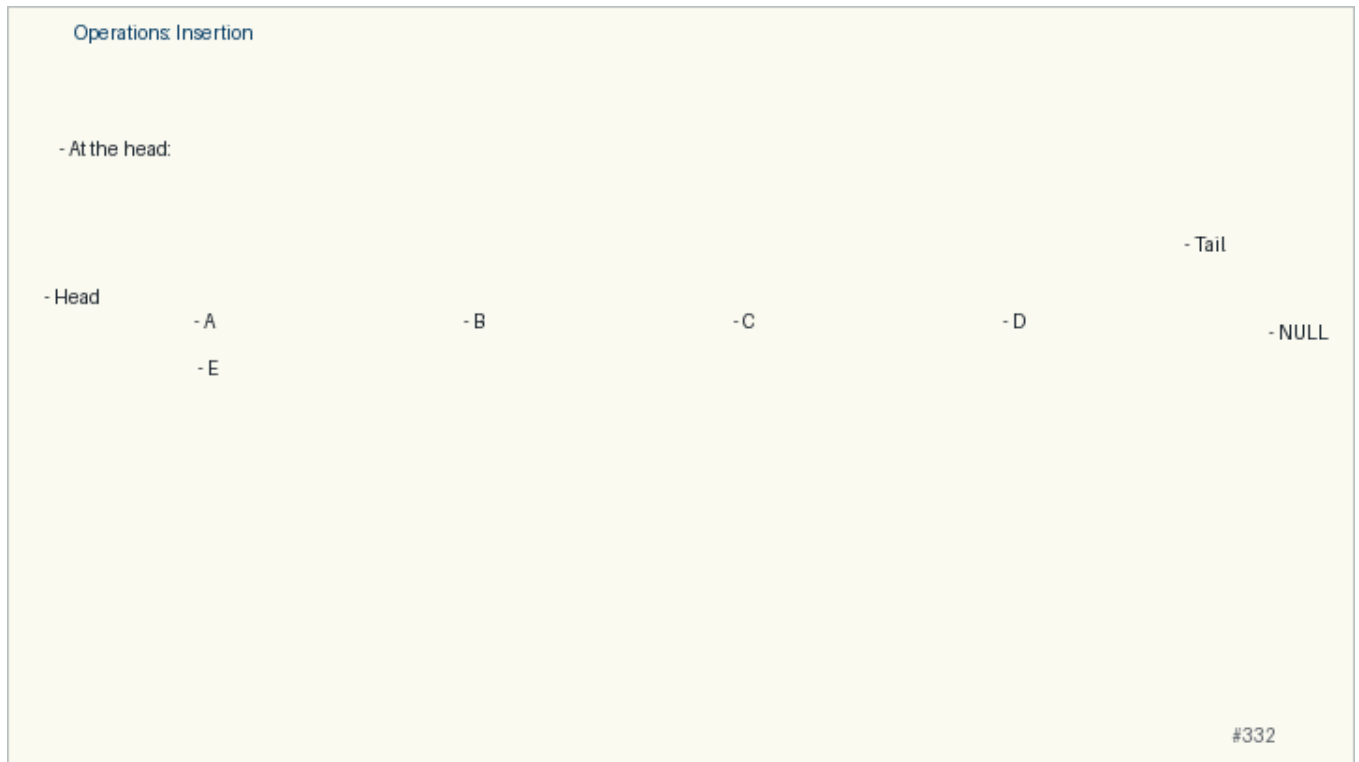
- !!!
- At the head:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E
- ???



3.25 Backup: Operations: Insertion

Original slide 332

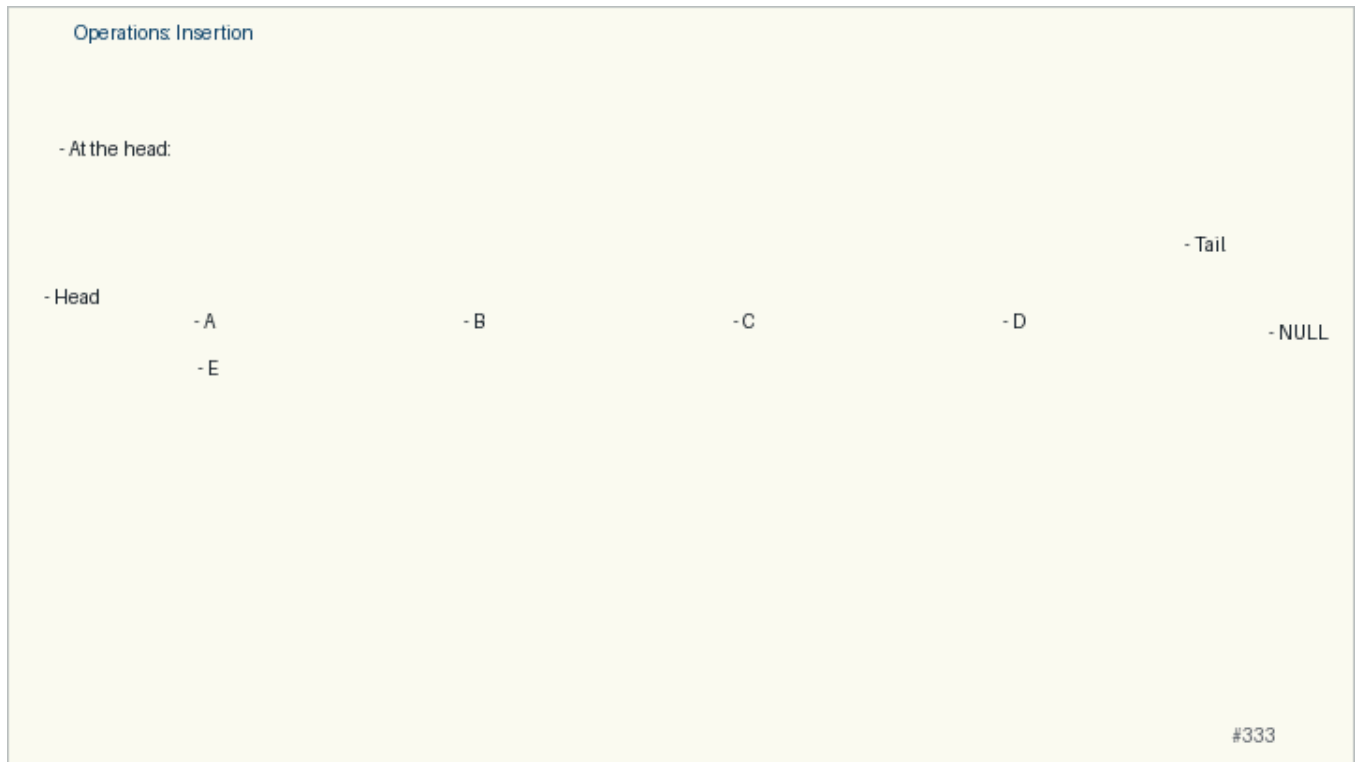
- At the head:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.26 Backup: Operations: Insertion

Original slide 333

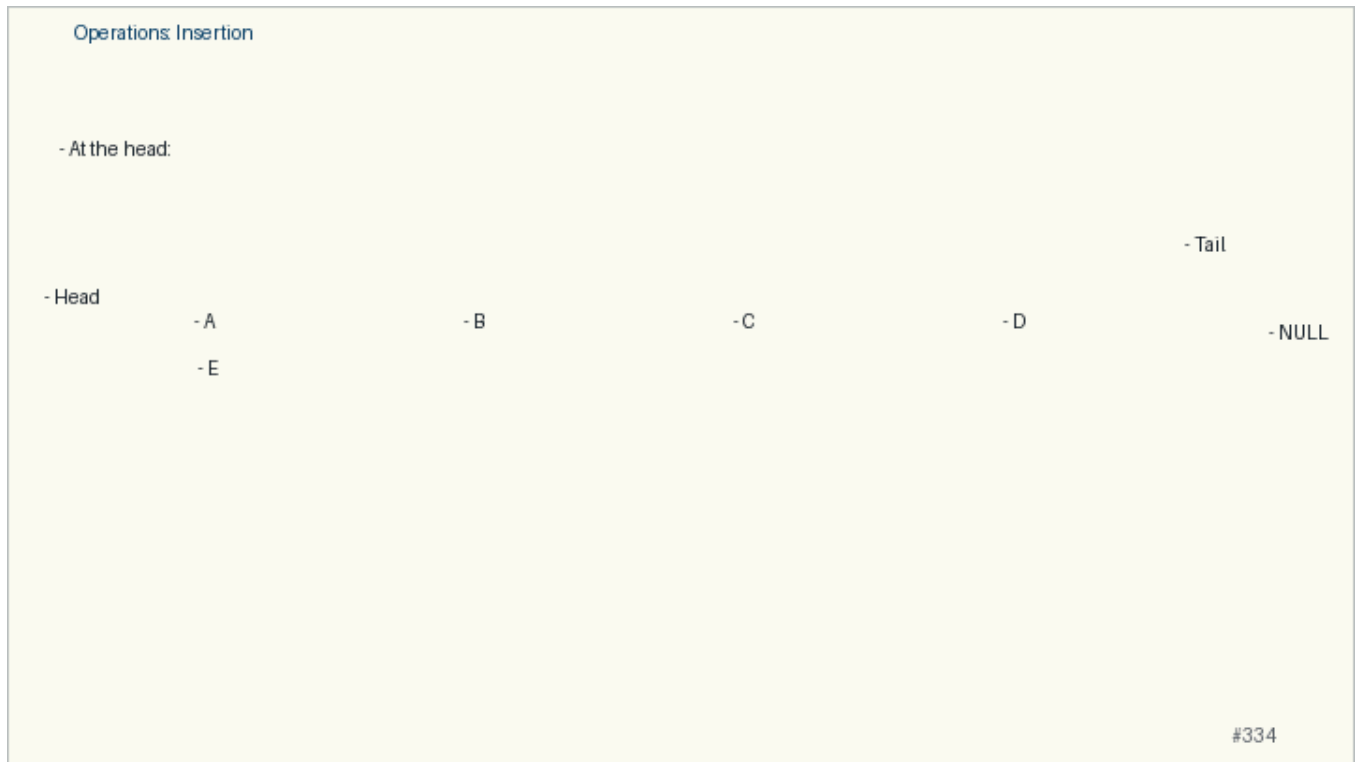
- At the head:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.27 Backup: Operations: Insertion

Original slide 334

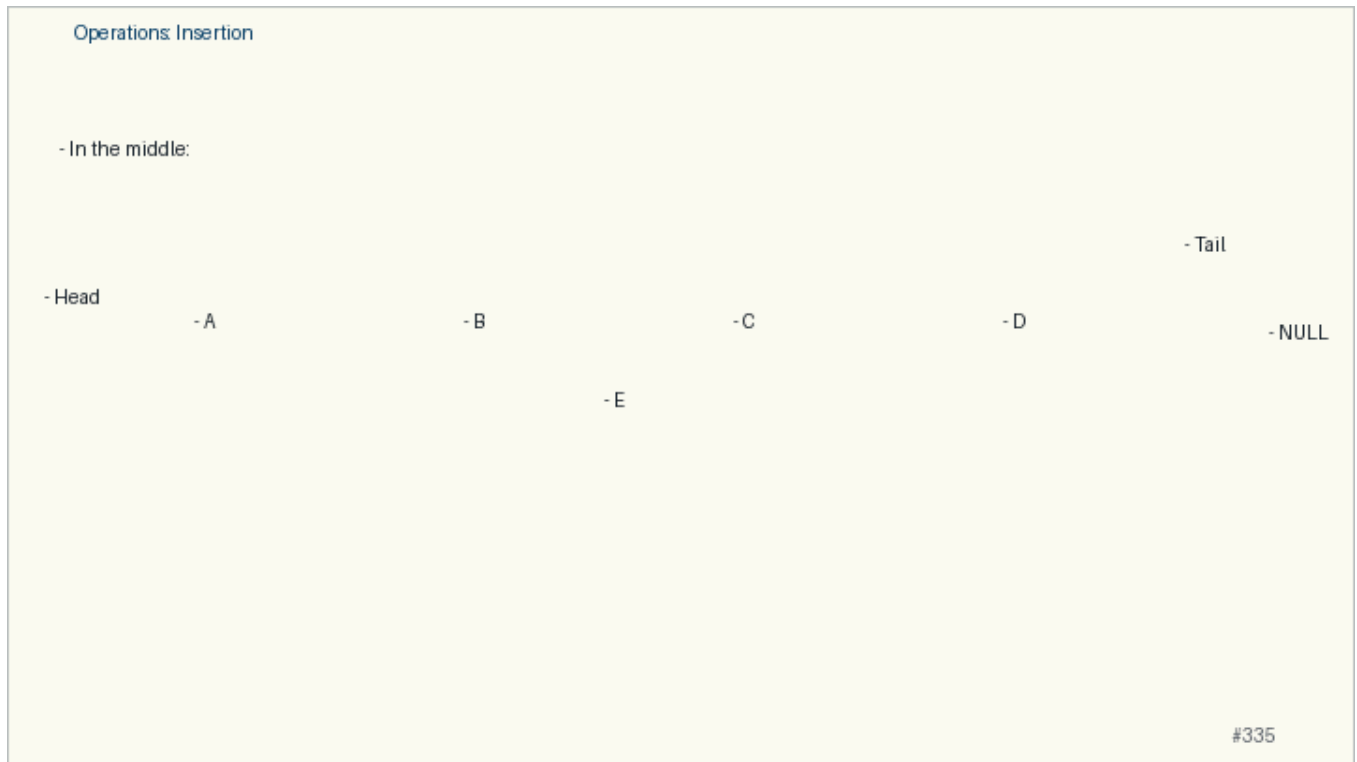
- At the head:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.28 Backup: Operations: Insertion

Original slide 335

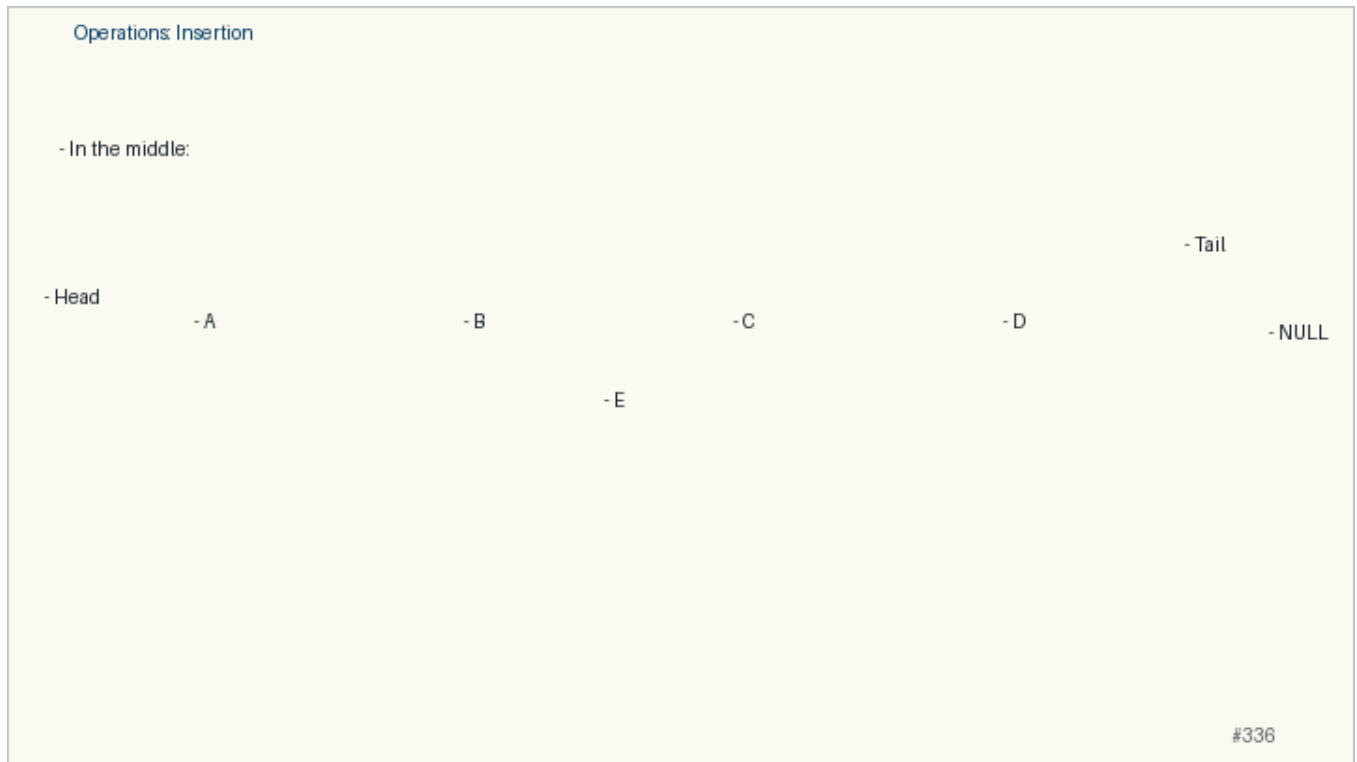
- In the middle:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.29 Backup: Operations: Insertion

Original slide 336

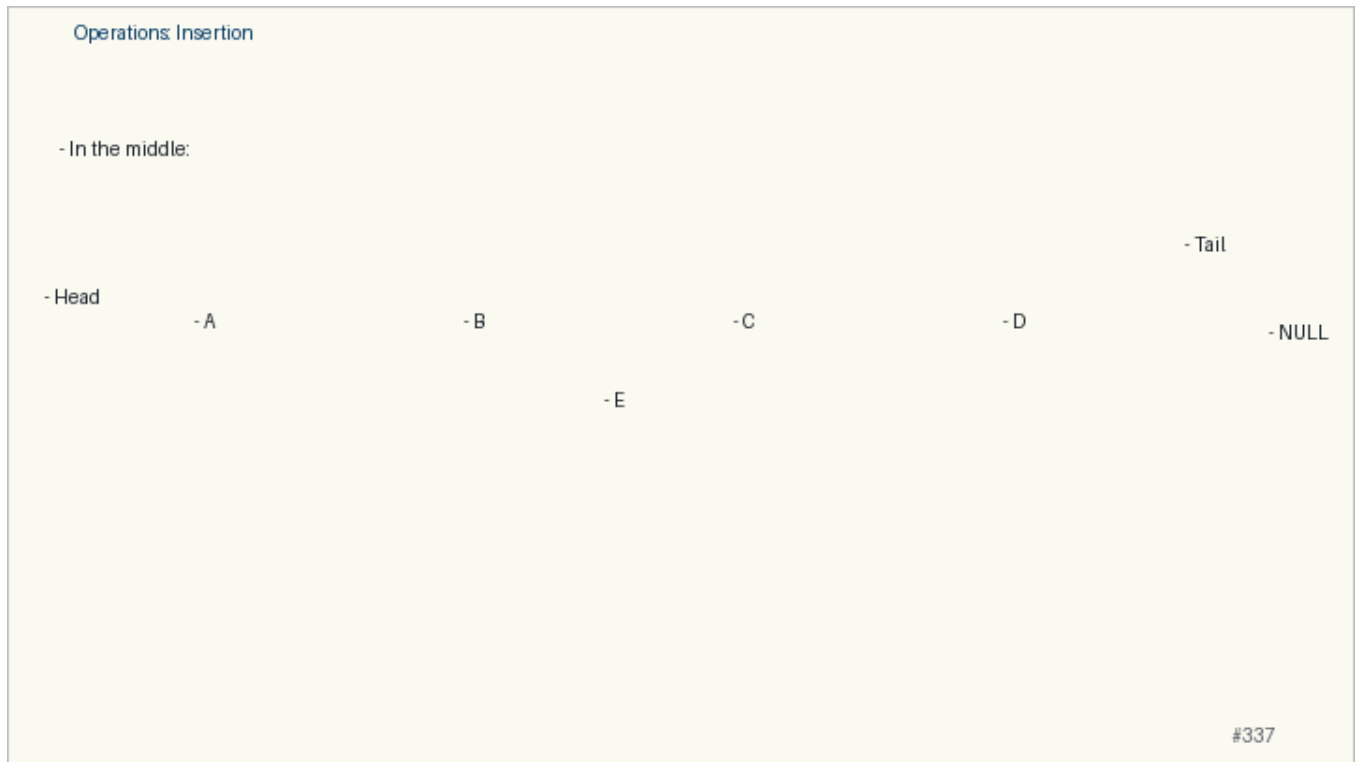
- In the middle:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.30 Backup: Operations: Insertion

Original slide 337

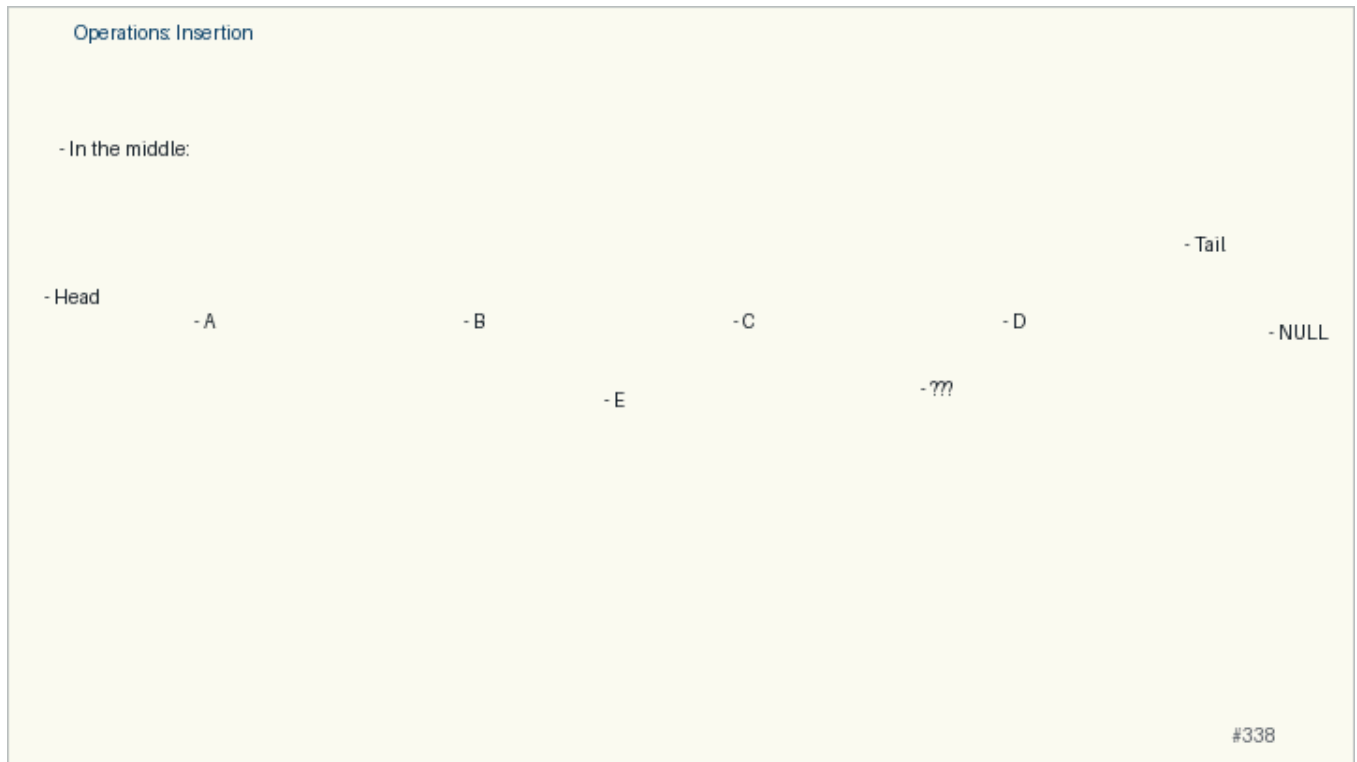
- In the middle:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.31 Backup: Operations: Insertion

Original slide 338

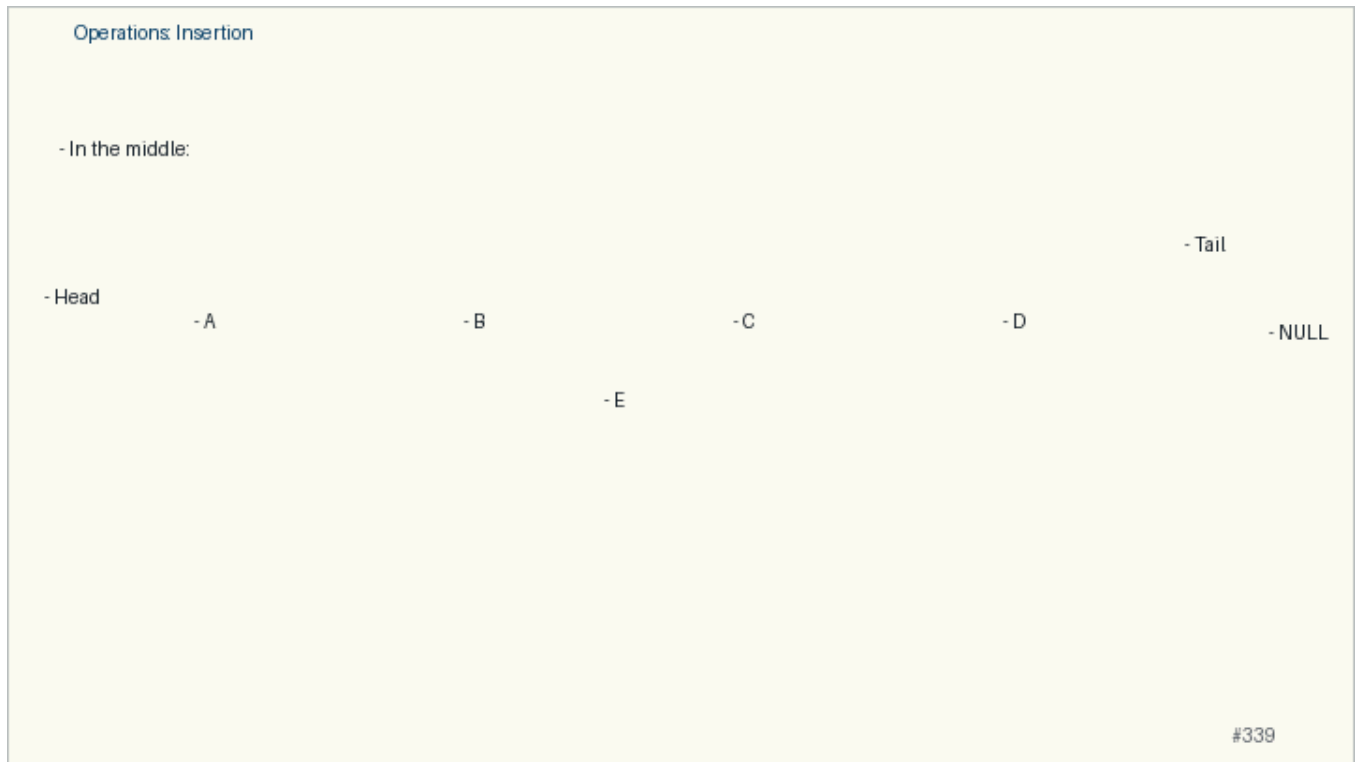
- In the middle:
- Tail
- Head
- A
- B
- C
- D
- NULL
- ???
- E



3.32 Backup: Operations: Insertion

Original slide 339

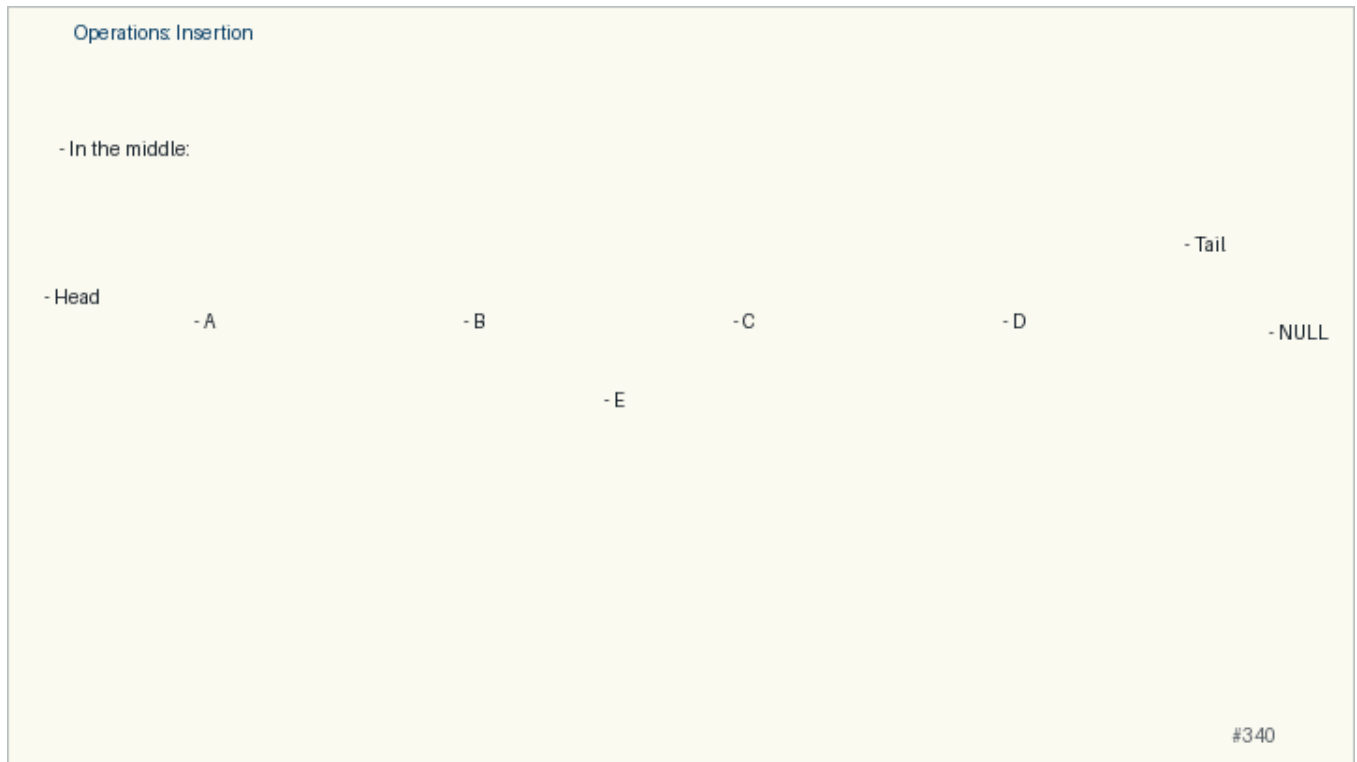
- In the middle:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.33 Backup: Operations: Insertion

Original slide 340

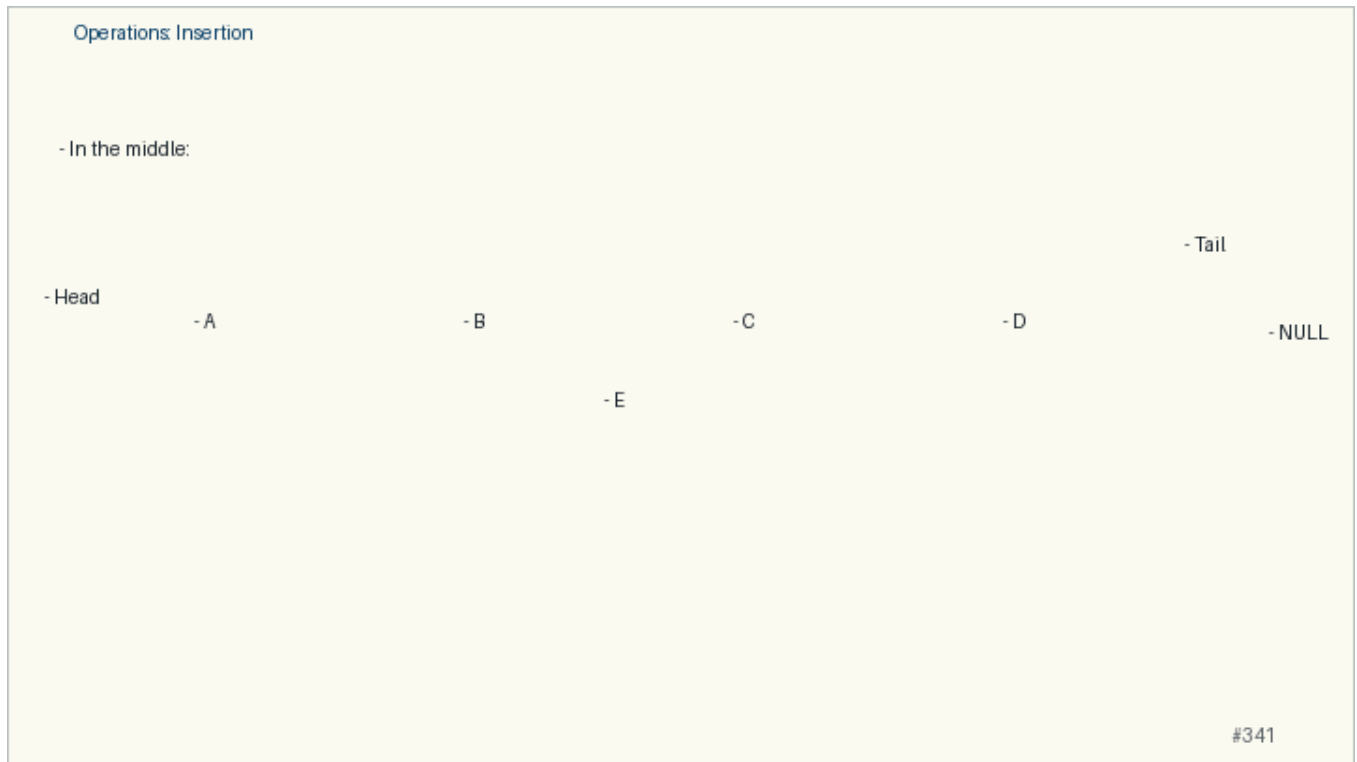
- In the middle:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.34 Backup: Operations: Insertion

Original slide 341

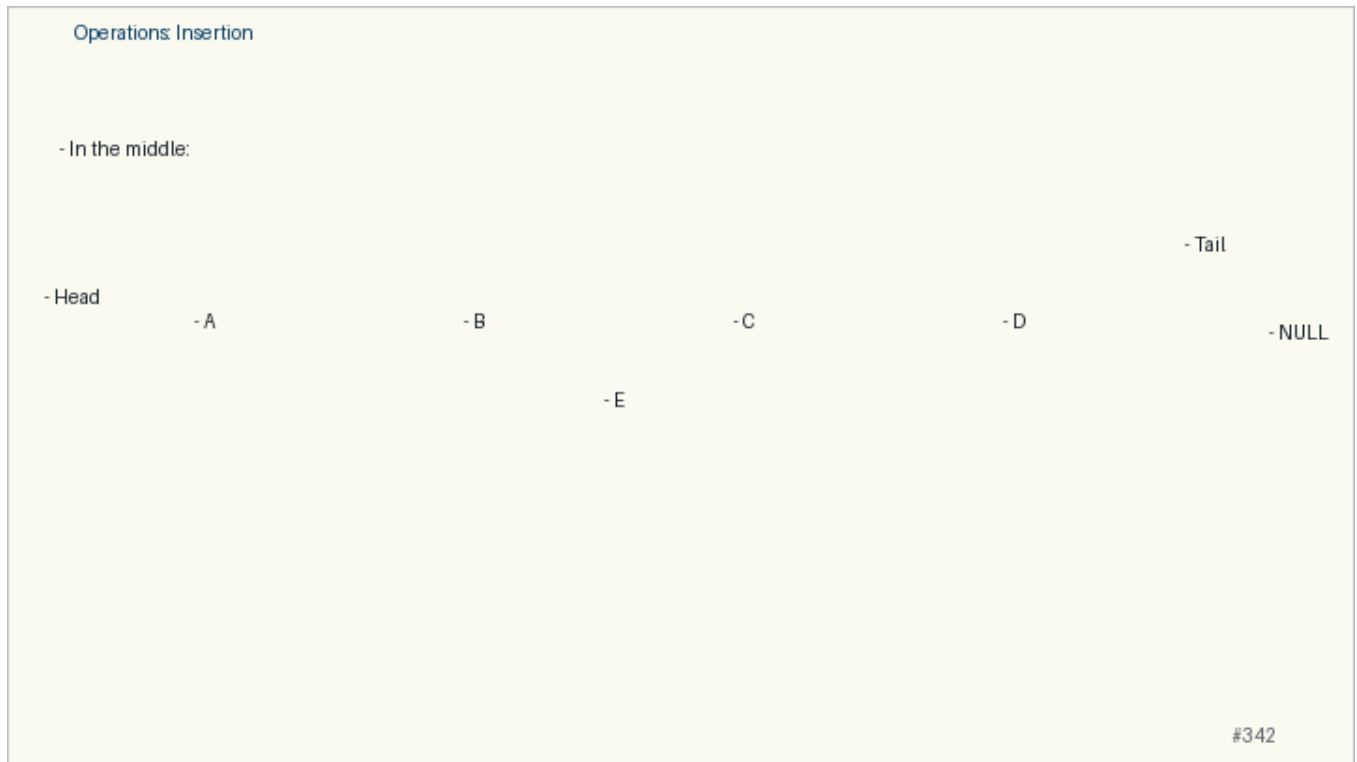
- In the middle:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.35 Backup: Operations: Insertion

Original slide 342

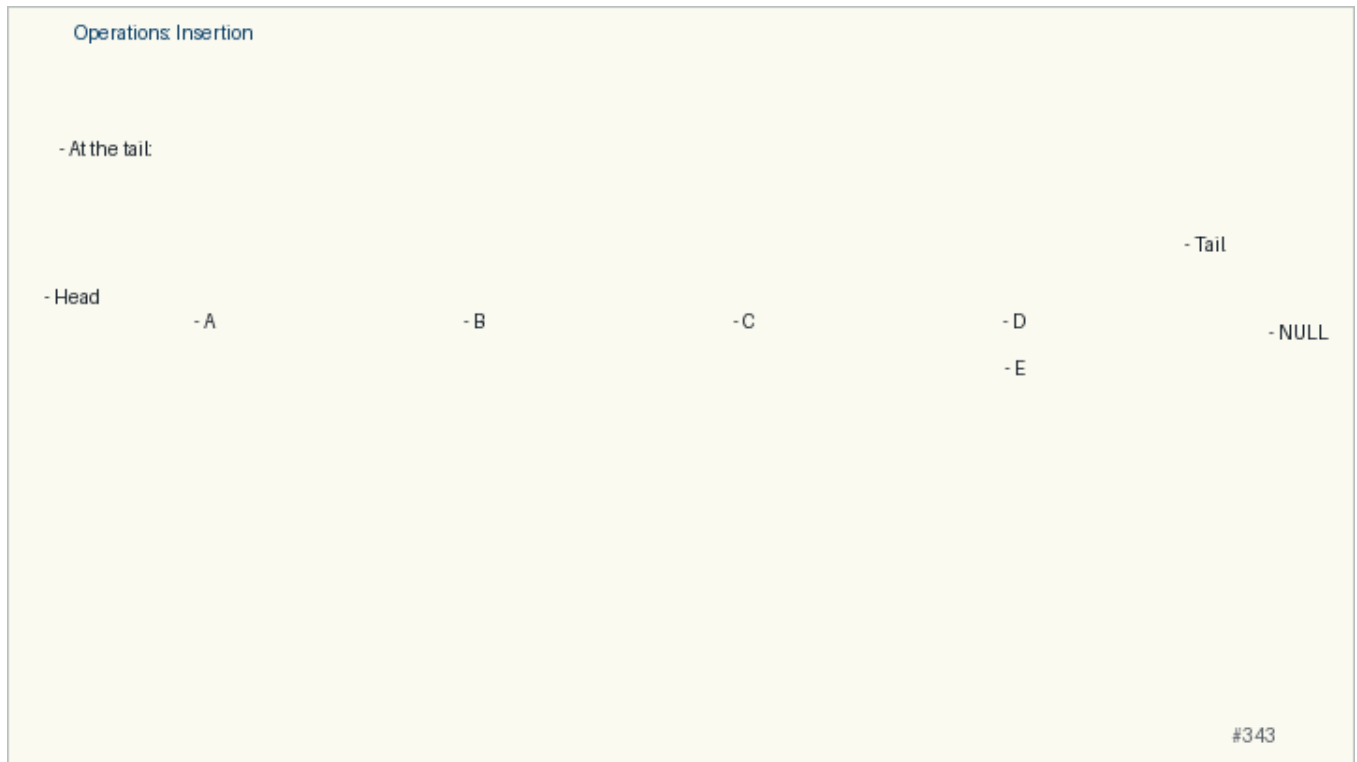
- In the middle:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.36 Backup: Operations: Insertion

Original slide 343

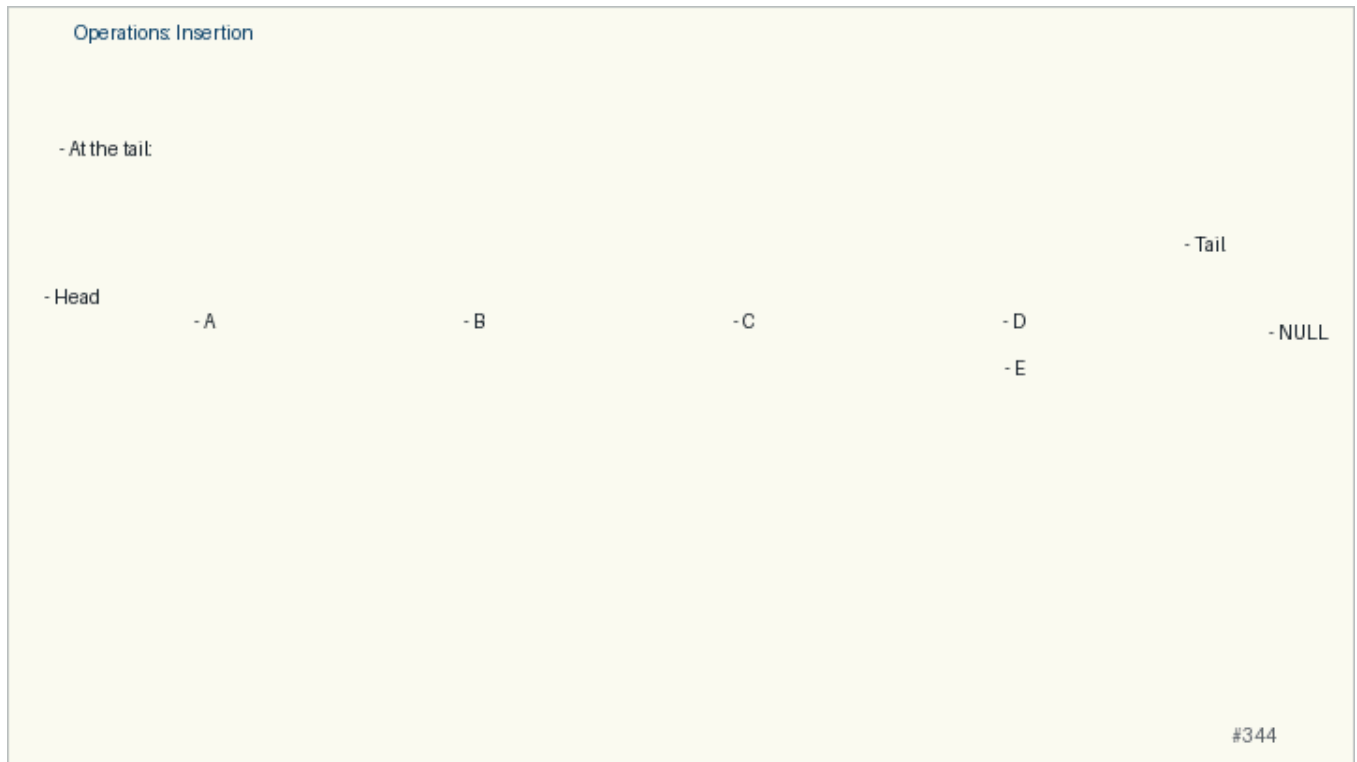
- At the tail:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.37 Backup: Operations: Insertion

Original slide 344

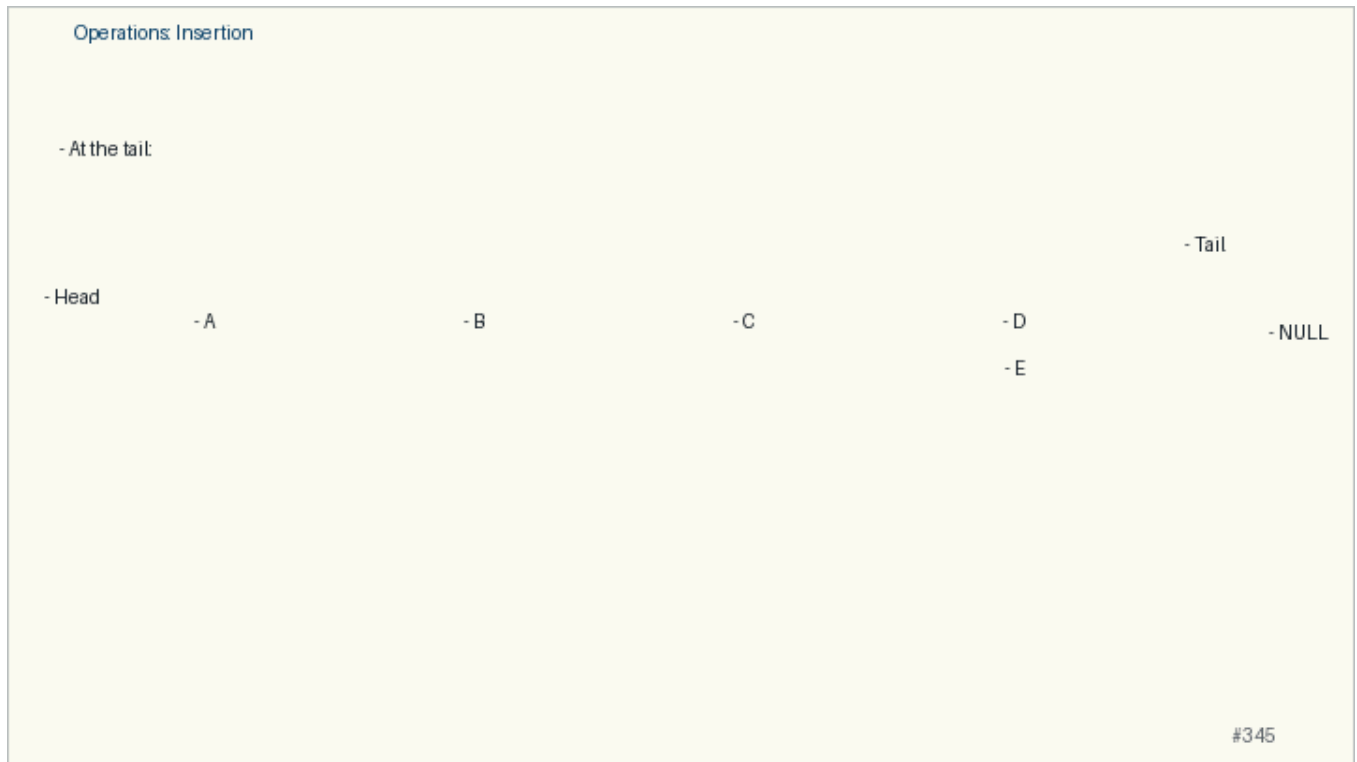
- At the tail:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.38 Backup: Operations: Insertion

Original slide 345

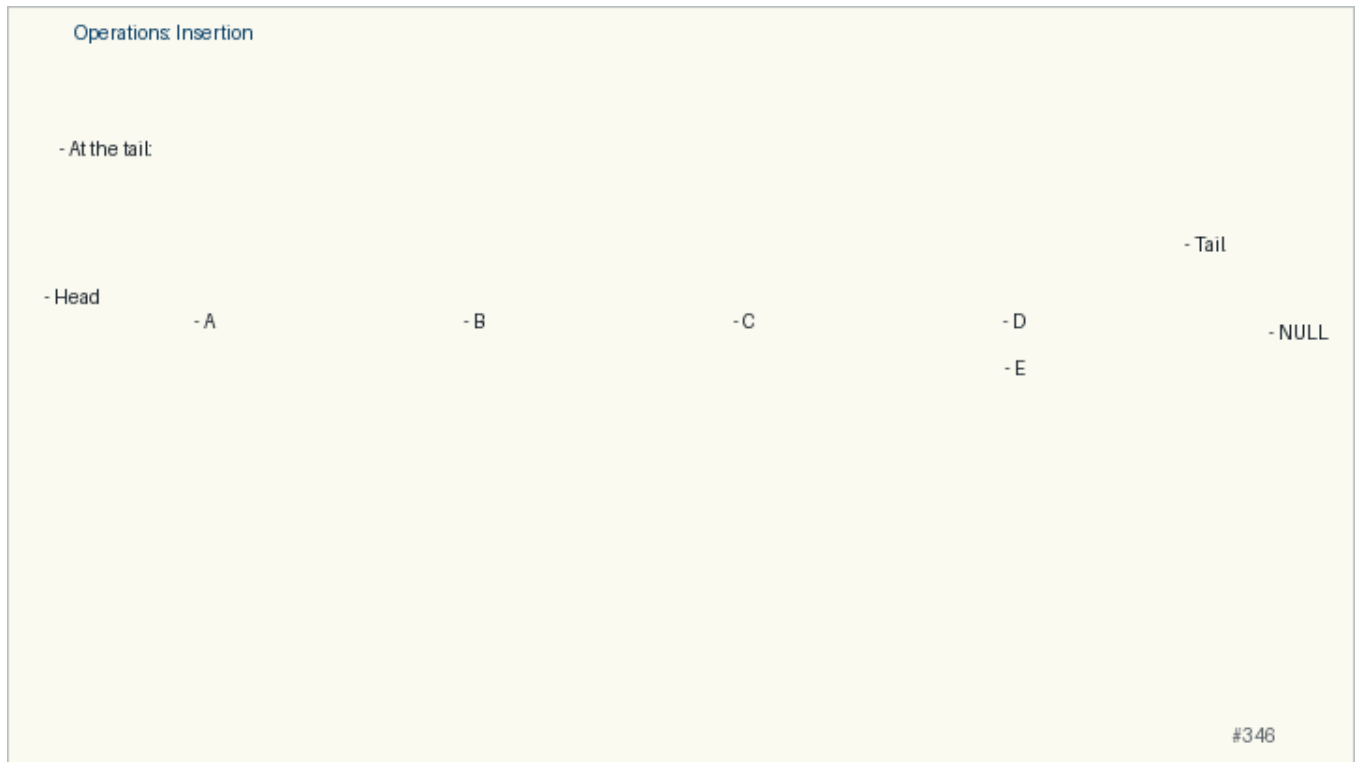
- At the tail:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.39 Backup: Operations: Insertion

Original slide 346

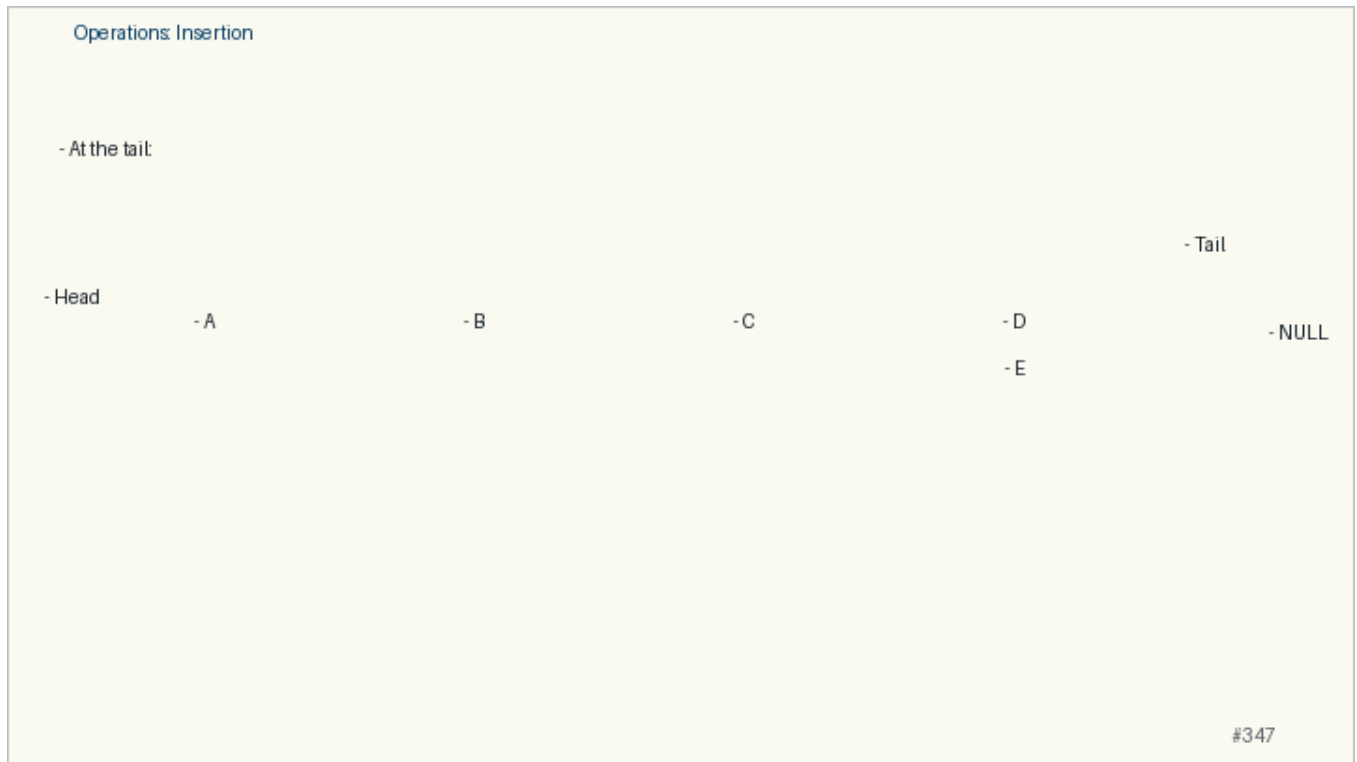
- At the tail:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.40 Backup: Operations: Insertion

Original slide 347

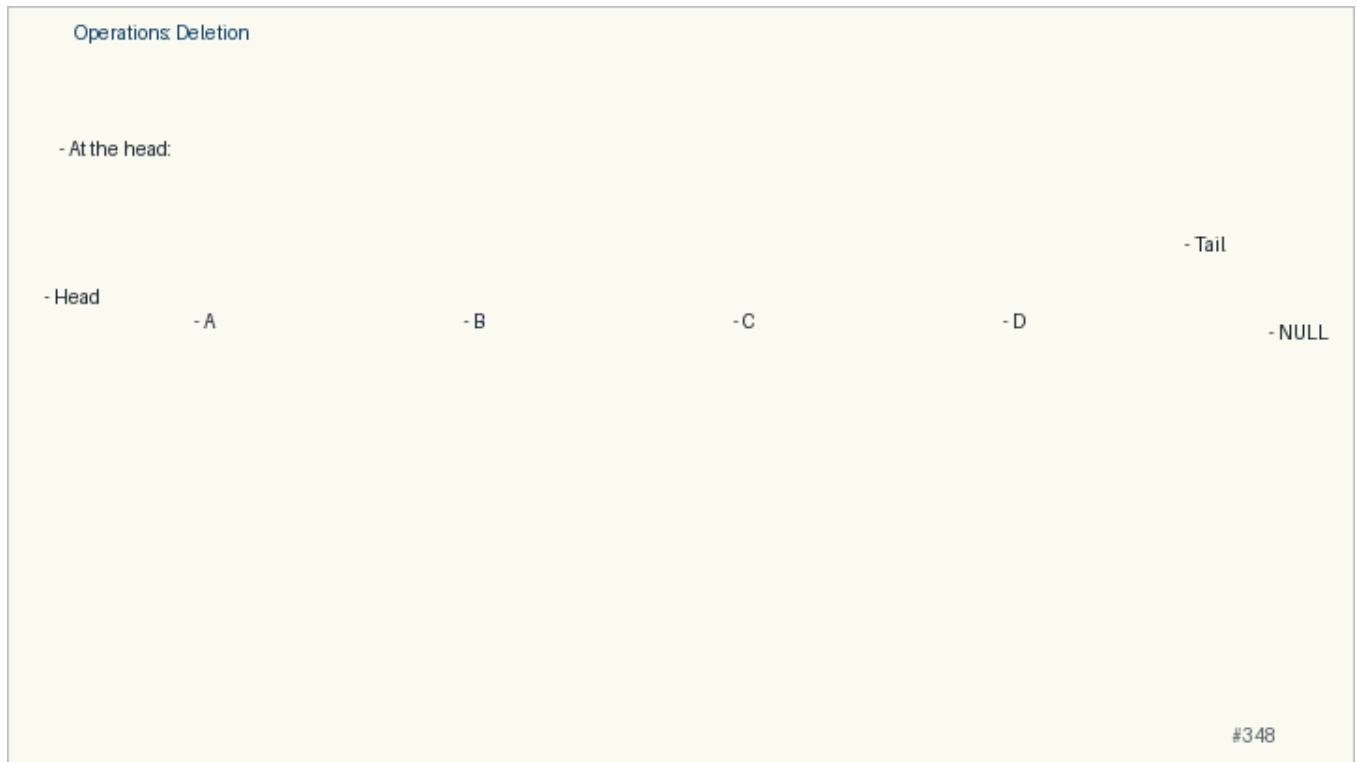
- At the tail:
- Tail
- Head
- A
- B
- C
- D
- NULL
- E



3.41 Backup: Operations: Deletion

Original slide 348

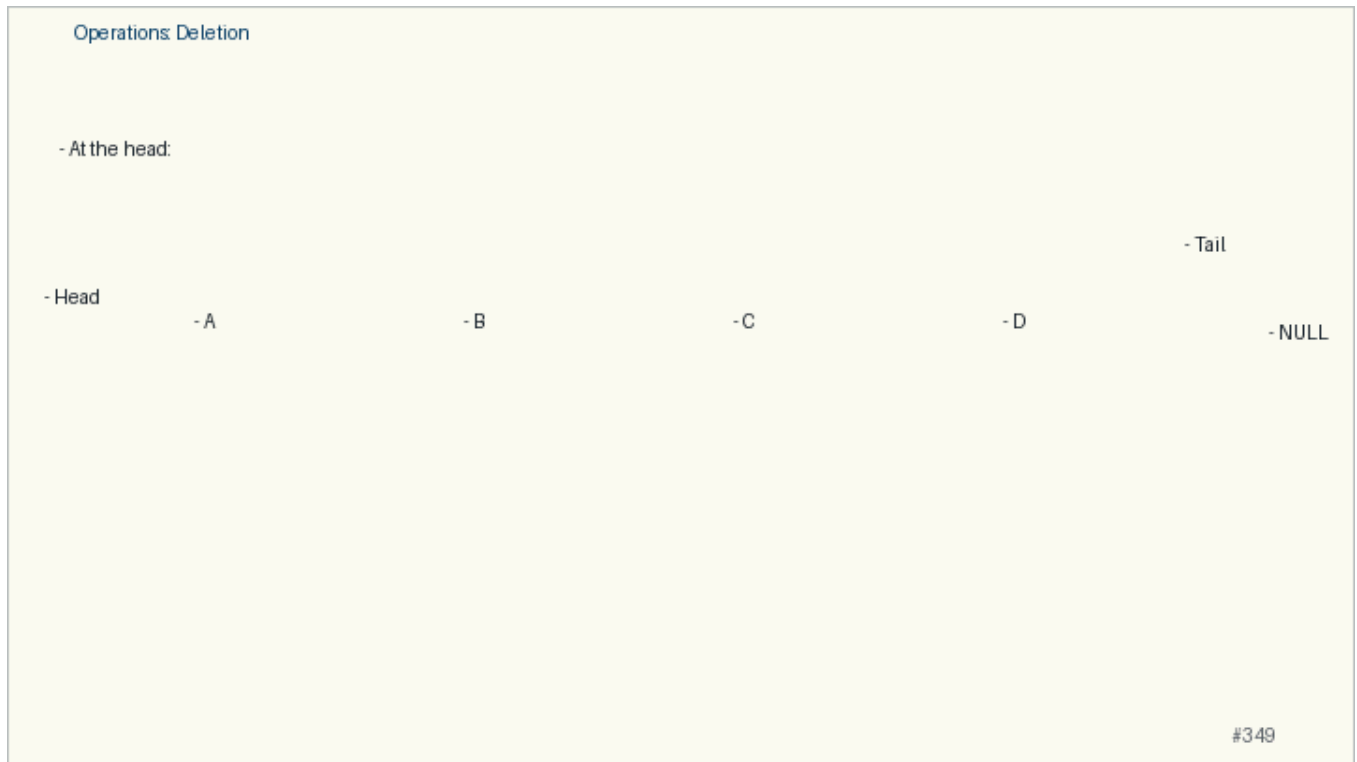
- At the head:
- Tail
- Head
- A
- B
- C
- D
- NULL



3.42 Backup: Operations: Deletion

Original slide 349

- At the head:
- Tail
- Head
- A
- B
- C
- D
- NULL



3.43 Backup: Operations: Deletion

Original slide 350

- At the head:
- Tail
- Head
- A
- B
- C
- D
- NULL
- ATTENTION: Free the memory space used for element A
- Need to know the pointer to the element A!

Operations: Deletion

- At the head:

- Tail

- Head

- A

- B

- C

- D

- NULL

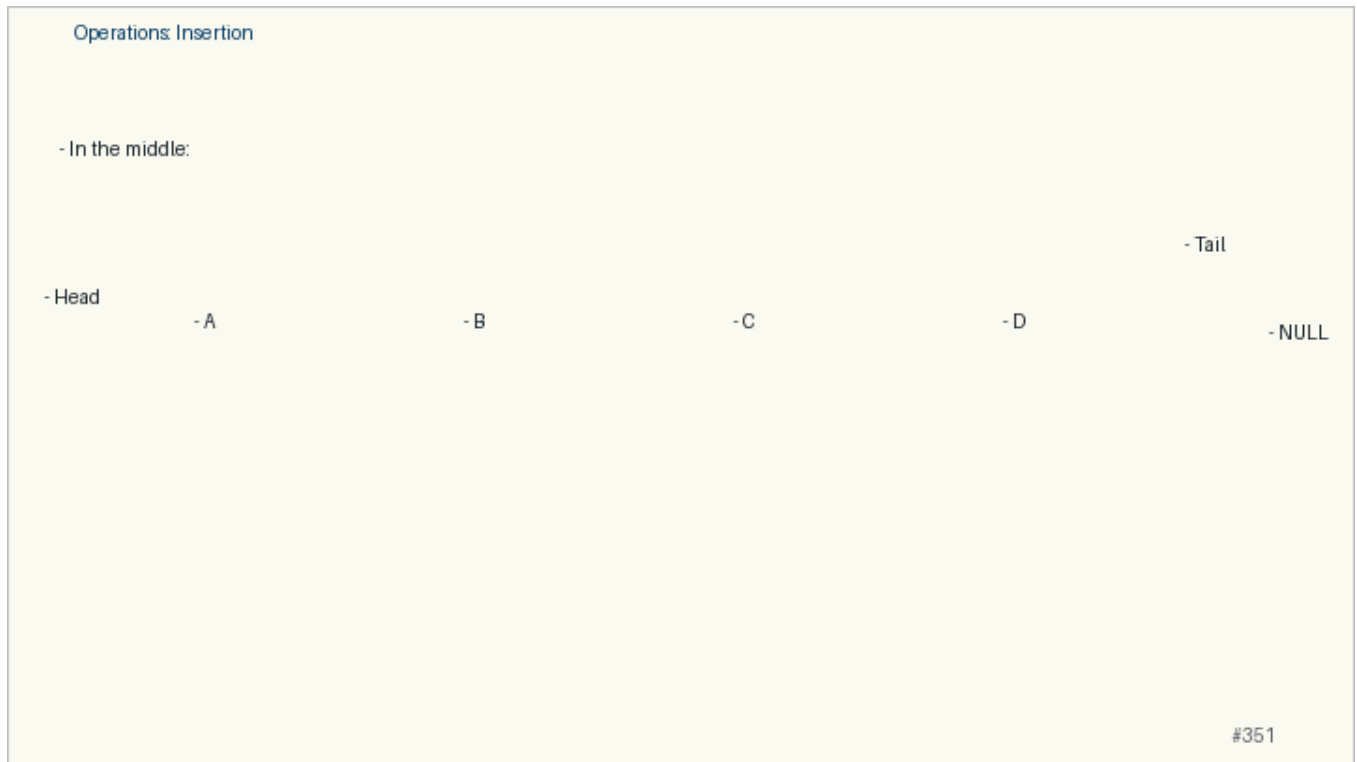
- ATTENTION: Free the memory space used for element A
- Need to know the pointer to the element A!

#350

3.44 Backup: Operations: Insertion

Original slide 351

- In the middle:
- Tail
- Head
- A
- B
- C
- D
- NULL



3.45 Backup: Operations: Insertion

Original slide 352

- In the middle:
- Tail
- Head
- A
- B
- C
- D
- NULL
- ATTENTION: Free the memory space used for element B
- Need to know the pointer to the element B!

Operations: Insertion

- In the middle:

- Tail

- Head

- A

- B

- C

- D

- NULL

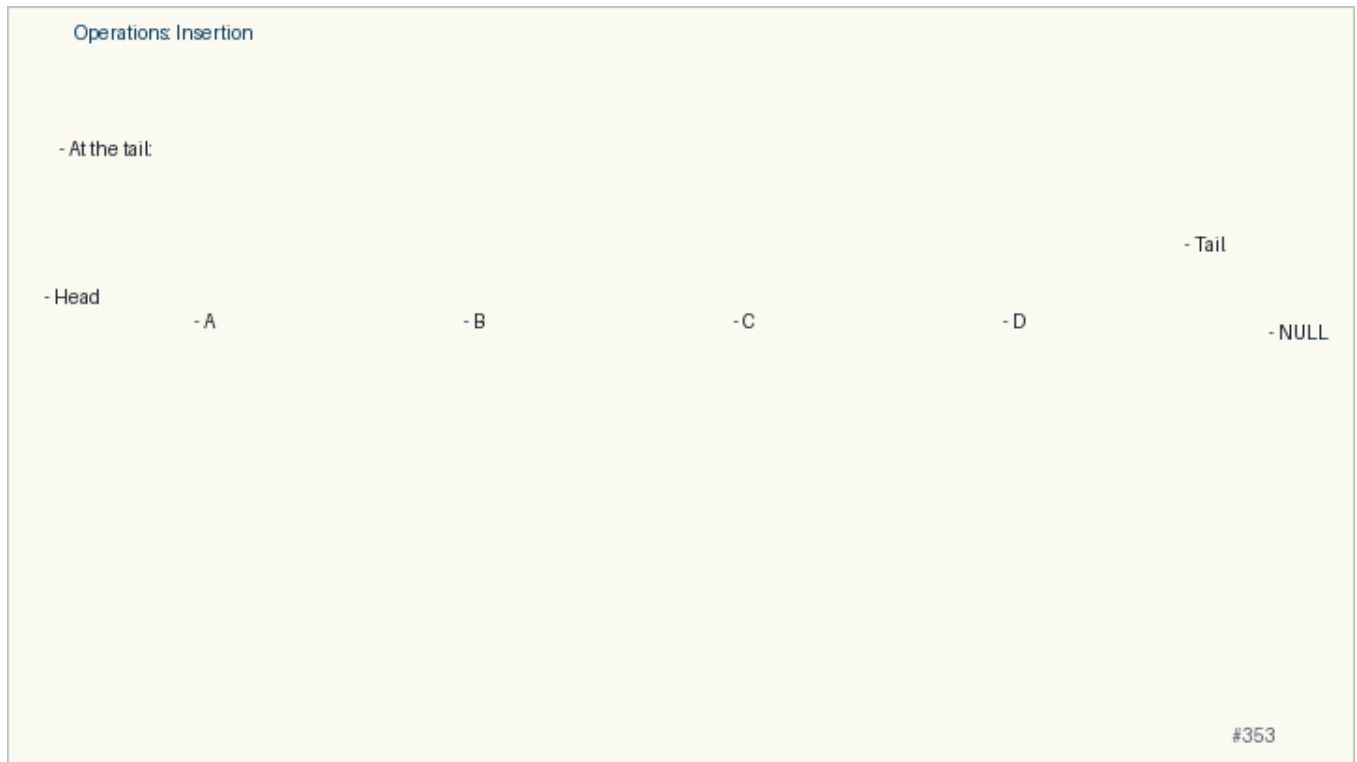
- ATTENTION: Free the memory space used for element B
- Need to know the pointer to the element B!

#352

3.46 Backup: Operations: Insertion

Original slide 353

- At the tail:
- Tail
- Head
- A
- B
- C
- D
- NULL



3.47 Backup: Operations: Insertion

Original slide 354

- At the tail:
- Tail
- Head
- A
- B
- C
- D
- NULL
- ATTENTION: Free the memory space used for element D
- Need to know the pointer to the element C!

Operations: Insertion

- At the tail:

- Tail

- Head

- A

- B

- C

- D

- NULL

- ATTENTION: Free the memory space used for element D
- Need to know the pointer to the element C!

#354

3.48 Backup: Doubly linked list

Original slide 355

- element

```
Each element has
data
prev: Link to the previous element
next: Link to the next element
Advantages
Traversed both in forward and backward direction
Delete operation is more efficient, previous pointer is given
Singly list: We have to traverse the list to find it
Disadvantages:
Extra memory space for storing the previous pointer
All operations have to maintain this extra pointer.
```

- prev
- next
- data

Doubly linked list

Each element has

data

prev. Link to the previous element

next. Link to the next element

Advantages

Traversed both in forward and backward direction

Delete operation is more efficient, previous pointer is given

Singly list We have to traverse the list to find it

Disadvantages

Extra memory space for storing the previous pointer

All operations have to maintain this extra pointer.

- element

- prev

- next

- data

#355

4 Hash Tables

Live pacing: about 11 min

Question. When stack objects are not enough, who owns memory and structure?

Core claim. Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.

Target capability. Students can pair allocation/free, define structs, and reason about linked/hash structures.

4.1 Main Path

- Start from the question: When stack objects are not enough, who owns memory and structure?
- Build the model: Dynamic allocation gives control over lifetime, but ownership must be made explicit in the design.
- End with a checkable action: Students can pair allocation/free, define structs, and reason about linked/hash structures.
- Keep only this slide on the horizontal path during the live lecture.

- Hash Table

- An array of a fixed size hsize
- An element is associated with an integer value called hash value
- a number within the range 0 to hsize - 1
- depicts in which cell of the hash table the element will be stored (key)
- calculated using a Hash Function
- Maps a data set of arbitrary size to a data set of fixed size
- unique value
- Fast access of the data:
- Search in the index indicated by the key

#356

4.2 Worked Example

- Use this as the live trace: Allocate one node, link it into a list, then remove it and state exactly who frees it.
- Representative source slide: 366
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

- Hash table: Dictionary

```
hash_table *htable_ptr;  
htable_ptr = (hash_table *) malloc(sizeof(hash_table));
```

Hash table allocation:
malloc reserves a memory space equal to the structure hash_table
htable_ptr will point to a structure containing an integer and a pointer to a word_list

- Hash table size initialisation
- We have finished the hash table declaration?
 - htable_ptr->hsize = PRIME_NUMBER

#366

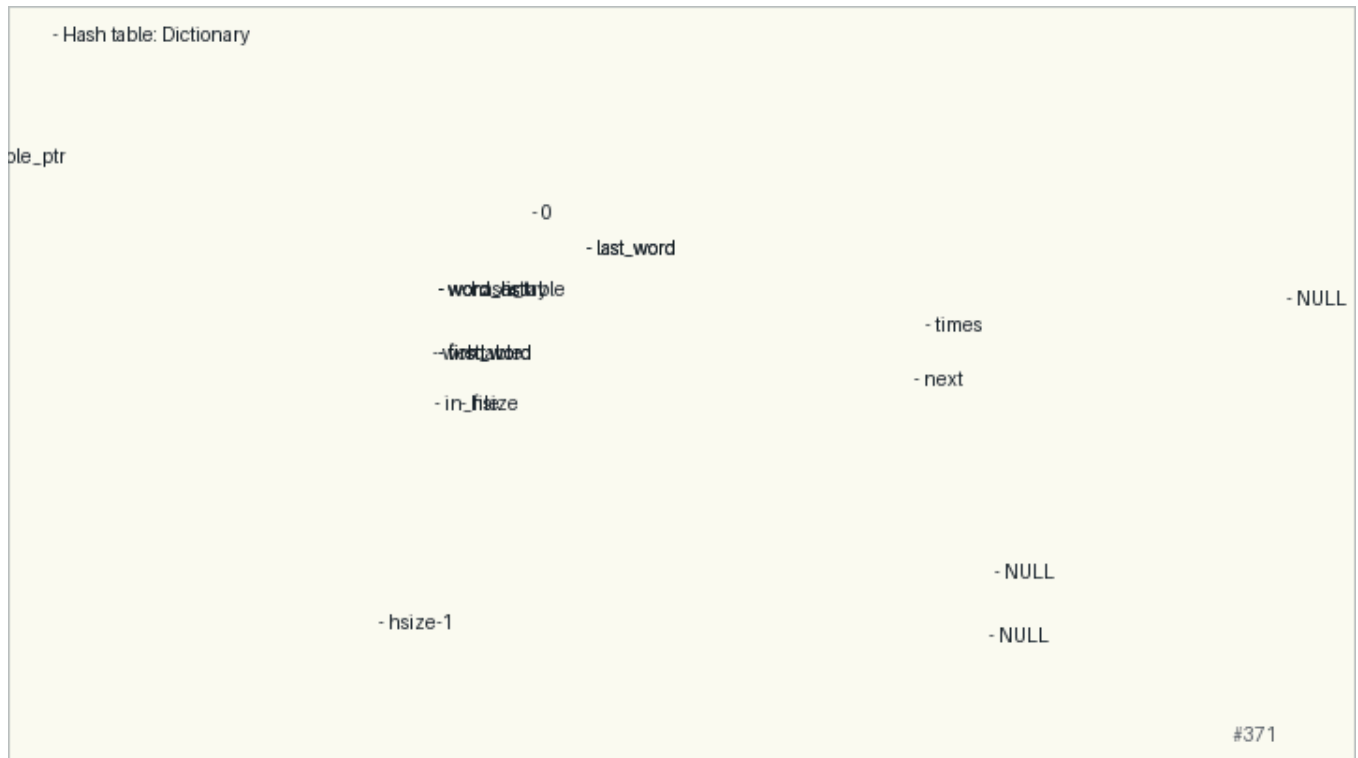
4.3 Anecdote Or Motivation

Every `malloc` creates a future obligation. Good C code makes that obligation visible.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

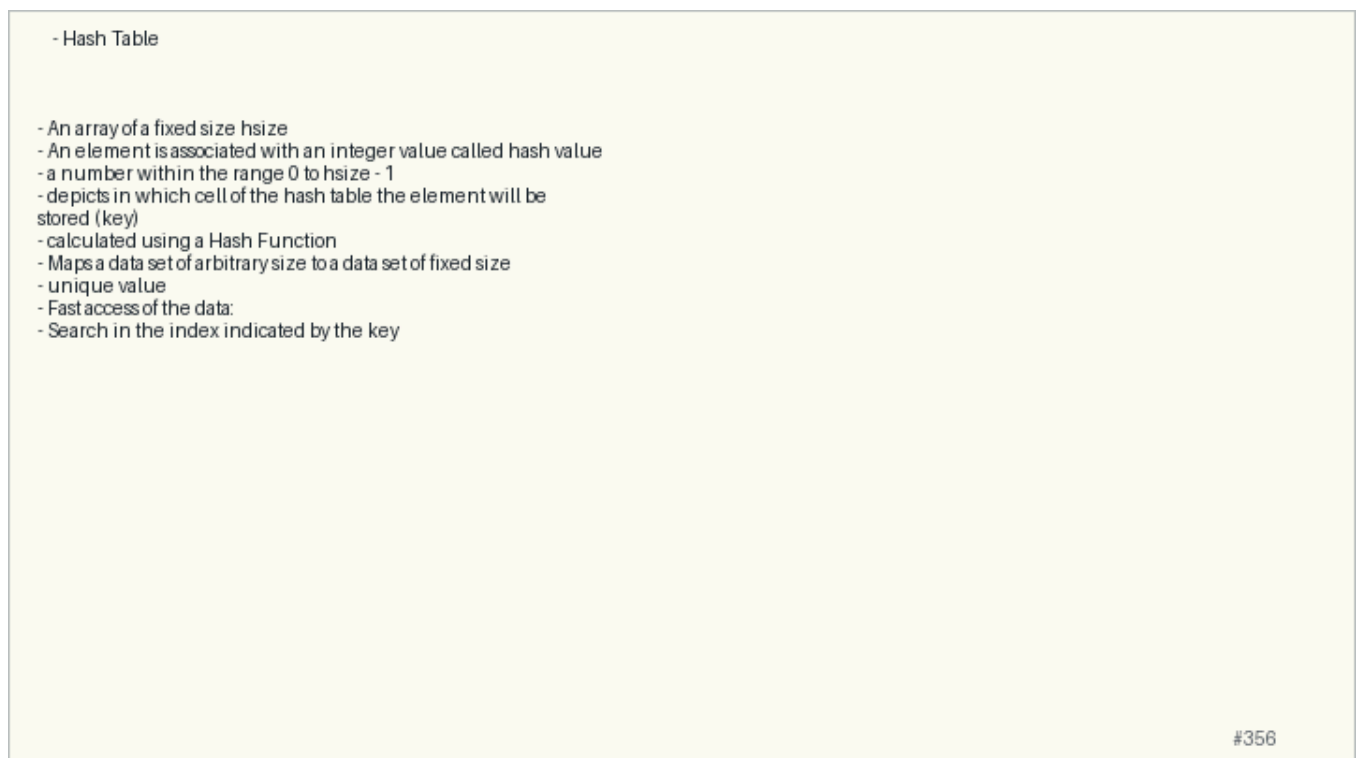
4.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.



4.5 Backup Index

Original slides 356-371



slide 356

- Hash Table: Some of its use

- Caches: Data tables used to speed up the data
- Object representation: Dynamic languages (Perl, Python, JavaScript) use hash tables to implement objects
- Algorithms to make computing faster
- Database indexing: In disk-based data tables and database indexes

#357

slide 357

- Hash Function: Simple examples

Integers:
Return key MOD hsize
Strings:
We add up the ASCII values of the chars, and return the modulo operation

```
int hash( char *key, int hsize )  
{  
    int hash_val = 0;  
    while( *key != '\0' )  
        hash_val += *key++;  
    return( hash_val % hsize );  
}
```

#358

slide 358

- Collisions

- When more than one element have the same hash value !
- Example:
- hsize = 10
- Elements: the first 10 perfect squares
- 0, 1, 4, 9, 16, 25, 36, 49, 64, 81
- hash function: $\text{hash}(x) = x \bmod 10$

#359

slide 359

- Collisions: Solution

- Separate chaining (open hashing):
- Each element of the hash table is a linked list
- Collision: Store both elements in the same linked list

- 0	- NULL	
- 1	- 81	- NULL
- 4	- 64	- NULL
- 25	- NULL	
- 16	- 36	- NULL
- 9	- 49	- NULL

#360

slide 360

- Problems

The distribution of the values may NOT be even !

Example

hsize = 10007

Variable: strings of 8 ASCII characters

The hash function: adds up the ASCII values of the chars and return the modulo operation

-> it can only assume values 0 - 1016, which is $127 * 8$

#361

slide 361

Anchors: Hash Table; Hash Table: Some of its use; Hash Function: Simple examples; Collisions; Collisions: Solution; Problems

4.6 Backup: Hash Table

Original slide 356

- An array of a fixed size hsize
- An element is associated with an integer value called hash value
- a number within the range 0 to hsize - 1
- depicts in which cell of the hash table the element will be stored (key)
- calculated using a Hash Function
- Maps a data set of arbitrary size to a data set of fixed size
- unique value
- Fast access of the data:
- Search in the index indicated by the key

- Hash Table

- An array of a fixed size hsize
- An element is associated with an integer value called hash value
- a number within the range 0 to hsize - 1
- depicts in which cell of the hash table the element will be stored (key)
- calculated using a Hash Function
- Maps a data set of arbitrary size to a data set of fixed size
- unique value
- Fast access of the data:
- Search in the index indicated by the key

#356

4.7 Backup: Hash Table: Some of its use

Original slide 357

- Caches: Data tables used to speed up the data
- Object representation: Dynamic languages (Perl, Python, JavaScript) use hash tables to implement objects
- Algorithms to make computing faster
- Database indexing: In disk-based data tables and database indexes

- Hash Table: Some of its use

- Caches: Data tables used to speed up the data
- Object representation: Dynamic languages (Perl, Python, JavaScript) use hash tables to implement objects
- Algorithms to make computing faster
- Database indexing: In disk-based data tables and database indexes

#357

4.8 Backup: Hash Function: Simple examples

Original slide 358

Integers:

Return key MOD hsize

Strings:

We add up the ASCII values of the chars, and **return** the modulo operation

```
int hash( char *key, int hsize )
{
    int hash_val = 0;
    while( *key != '\0' )
        hash_val += *key++;
    return( hash_val % hsize );
}
```

- Hash Function: Simple examples

Integers:

Return key MOD hsize

Strings:

We add up the ASCII values of the chars, and return the modulo operation

```
int hash( char *key, int hsize )
{
    int hash_val = 0;
    while( *key != '\0' )
        hash_val += *key++;
    return( hash_val % hsize );
}
```

#358

4.9 Backup: Collisions

Original slide 359

- When more than one element have the same hash value !
- Example:
- hsize = 10
- Elements: the first 10 perfect squares
- 0, 1, 4, 9, 16, 25, 36, 49, 64, 81
- hash function: $\text{hash}(x) = x \bmod 10$

- Collisions

- When more than one element have the same hash value !
- Example:
- hsize = 10
- Elements: the first 10 perfect squares
- 0, 1, 4, 9, 16, 25, 36, 49, 64, 81
- hash function: $\text{hash}(x) = x \bmod 10$

#359

4.10 Backup: Collisions: Solution

Original slide 360

- Separate chaining (open hashing):
- Each element of the hash table is a linked list
- Collision: Store both elements in the same linked list
- 0
- NULL
- 1
- 81
- NULL
- 4
- 64
- NULL
- 25
- NULL
- 16
- 36
- NULL
- 9
- 49
- NULL

- Collisions: Solution

- Separate chaining (open hashing):
- Each element of the hash table is a linked list
- Collision: Store both elements in the same linked list

- 0	- NULL	
- 1	- 81	- NULL
- 4	- 64	- NULL
- 25	- NULL	
- 16	- 36	- NULL
- 9	- 49	- NULL

#360

4.11 Backup: Problems

Original slide 361

The distribution of the values may NOT be even !

Example

hsize = 10007

Variable: strings of 8 ASCII characters

The hash function: adds up the ASCII values of the chars and **return** the modulo operation

-> it can only assume values 0 - 1016, which is $127 * 8$

- Problems

The distribution of the values may NOT be even !

Example

hsize = 10007

Variable: strings of 8 ASCII characters

The hash function: adds up the ASCII values of the chars and return the modulo operation

-> it can only assume values 0 - 1016, which is $127 * 8$

#361

4.12 Backup: Hash table: Dictionary

Original slide 362

- Create a dictionary and search engine:
- Read files word by word
- Add to the dictionary one word found in a file and the number of times found in the file
- Search for a word in the dictionary
- Use hash table and linked lists to store the words with the same key

- Hash table: Dictionary

- Create a dictionary and search engine:
- Read files word by word
- Add to the dictionary one word found in a file and the number of times found in the file
- Search for a word in the dictionary
- Use hash table and linked lists to store the words with the same key

#362

4.13 Backup: Hash table: Dictionary

Original slide 363

```
struct word_entry
{
    char word[SIZE];
    int in_file;
    int times;
    struct word_entry *next;
};
typedef struct word_entry word_entry;
```

- word_entry
- times
- word
- next
- in_file
- The structure word_entry has as elements
- a word of size SIZE
- an integer in_file which is the identifier of the file where the word has been found

- an integer times which shows how many times the word exist in this file
- a pointer *next to a structure word_entry

- Hash table: Dictionary

```

struct word_entry
{
    char word[SIZE];
    int in_file;
    int times;
    struct word_entry *next;
};
typedef struct word_entry word_entry;
  
```

- word_entry
- times
- word
- next
- in_file

- The structure word_entry has as elements
- a word of size SIZE
- an integer in_file which is the identifier of the file where the word has been found
- an integer times which shows how many times the word exist in this file
- a pointer *next to a structure word_entry

#363

4.14 Backup: Hash table: Dictionary

Original slide 364

```

struct word_list
{
    struct word_entry *first_word;
    struct word_entry *last_word;
};
typedef struct word_list word_list;
  
```

- last_word
- word_list
- first_word
- The structure word_list has as elements
- a pointer *first_word to a structure word_entry
- a pointer *last_word to a structure word_entry

- Hash table: Dictionary

```
struct word_list
{
    struct word_entry *first_word;
    struct word_entry *last_word;
};
typedef struct word_list word_list;
```

- word_list

- last_word

- first_word

- The structure word_list has as elements
- a pointer *first_word to a structure word_entry
- a pointer *last_word to a structure word_entry

#364

4.15 Backup: Hash table: Dictionary

Original slide 365

```
struct hash_table
{
    word_list *htable;
    int hsize;
};
typedef struct hash_table hash_table;
```

- hash_table
- htable
- The hash_table structure contains
- a pointer htable to a type word_list
- A pointer to a pointer of the structure word_entry
- the fixed size hsize
- We have finished the hash table declaration?
- hsize

- Hash table: Dictionary

```
struct hash_table
{
    word_list *htable;
    int hsize;
};
typedef struct hash_table hash_table;
```

- hash_table

- htable

- The hash_table structure contains
- a pointer htable to a type word_list
- A pointer to a pointer of the structure word_entry
- the fixed size hsize
- We have finished the hash table declaration?

#365

4.16 Backup: Hash table: Dictionary

Original slide 366

```
hash_table *htable_ptr;
htable_ptr = (hash_table*) malloc (sizeof(hash_table));
```

Hash table allocation:

malloc reserves a memory space equal to the structure hash_table

htable_ptr will point to a structure containing an integer and a pointer to a word_list

- Hash table size initialisation
- We have finished the hash table declaration?
- htable_ptr->hsize = PRIME_NUMBER;

- Hash table: Dictionary

```
hash_table *htable_ptr;  
htable_ptr = (hash_table *) malloc (sizeof(hash_table));
```

Hash table allocation:
malloc reserves a memory space equal to the structure hash_table
htable_ptr will point to a structure containing an integer and a pointer to a word_list

- Hash table size initialisation
- We have finished the hash table declaration?
- htable_ptr->hsize = PRIME_NUMBER

#366

4.17 Backup: Hash table: Dictionary

Original slide 367

- htable_ptr
- hash_table
- htable
- hsize

- Hash table: Dictionary

table_ptr

- hash_table

- htable

- hsize

#367

4.18 Backup: Hash table: Dictionary

Original slide 368

- Hash table pointer initialisation:
- We need to allocate space for the linked lists.
- We use an array of structures word_list equal to the table size

```
htable_ptr->htable =
(word_list*) malloc(sizeof(word_list) * htable_ptr->hsize );
```

- We need to initialize the pointers of the structures word_list to NULL

```
for(i = 0; i < htable_ptr->hsize ; i++ ){
htable_ptr->htable[i].first_word = NULL;
htable_ptr->htable[i].last_word = NULL;
}
```

- Hash table: Dictionary

- Hash table pointer initialisation:
- We need to allocate space for the linked lists.
- We use an array of structures word_list equal to the table size

```
htable_ptr->htable =
(word_list*) malloc(sizeof(word_list) * htable_ptr->hsize );
```

- We need to initialize the pointers of the structures word_list to NULL

```
for(i = 0; i < htable_ptr->hsize ; i++ ){
htable_ptr->htable[i].first_word = NULL;
htable_ptr->htable[i].last_word = NULL;
}
```

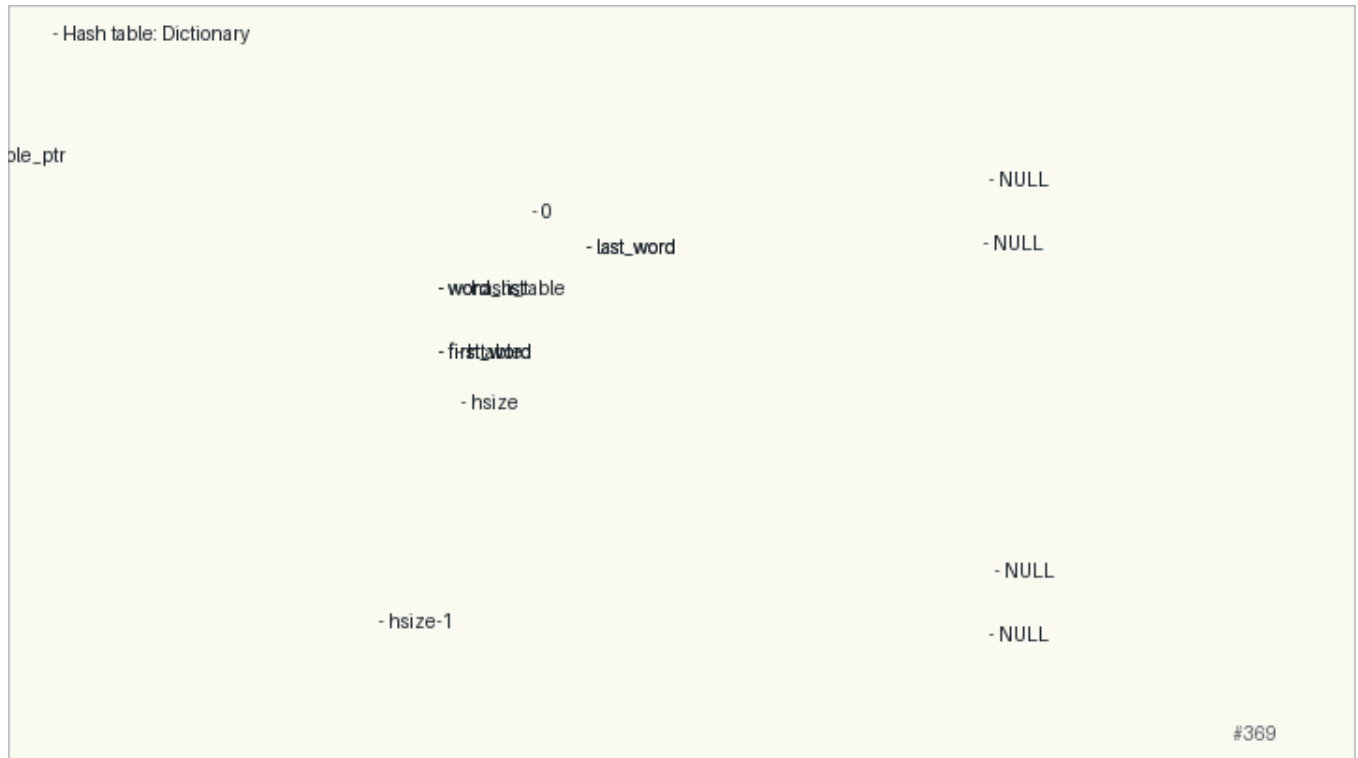
#368

4.19 Backup: Hash table: Dictionary

Original slide 369

- htable_ptr
- NULL
- 0
- NULL
- last_word
- last_word
- word_list
- word_list
- hash_table
- first_word
- first_word

- htable
- hsize
- NULL
- hsize-1
- NULL



4.20 Backup: Hash table: Dictionary

Original slide 370

- When a new word is added:
- We need to allocate space for the word
- Connected to the list based on the hash key

```
htable_ptr->htable[key].first_word =
(word_entry*)malloc(sizeof(word_entry));
```


- Hash table: Dictionary

- When a new word is added:
- We need to allocate space for the word
- Connected to the list based on the hash key

```
htable_ptr->htable[key].first_word =  
(word_entry*) malloc(sizeof(word_entry));
```

#370

4.21 Backup: Hash table: Dictionary

Original slide 371

- htable_ptr
- 0
- last_word
- last_word
- word_list
- word_entry
- word_list
- hash_table
- NULL
- times
- word
- first_word
- first_word
- htable
- next
- in_file
- hsize
- NULL
- hsize-1
- NULL

